SIAmese

Multi Object Tracking on “Visdrone” dataset

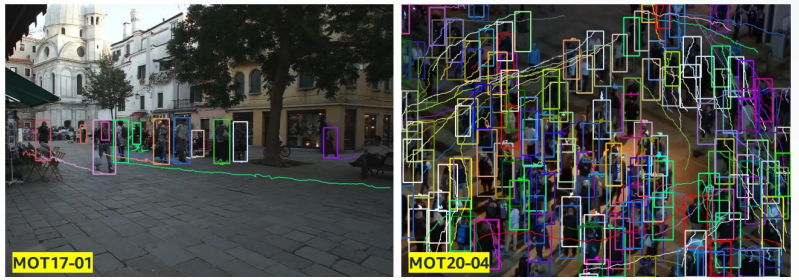
Ablation study on YOLO performance in multi object tracking

고민수, 손승우, 정민교

개요

프로젝트의 목표 및 목적

- 목표
  - ➢TASK
    - 드론, 항공 영상에서 나타나는 차량, 보행자의 위치를 식별하고 객체를 추적
  - ➢PERFORMANCE
    - 성능을 높일 수 있는 방안을 가정과 평가로 확인
- 목적
  - ➢TASK
    - 드론 기술의 발전에 따라 드론 영상 정보를 이용한 관련 시장 수요 증가
  - ➢PERFORMANCE
    - TASK 에 대해 더 높은 성능을 야기하는 요소 파악
    - 모델 구현능력 증진



MOT의 시각화 예시

*MOT: multi object tracking

팀 구성 및 역할

고민수

EDA
데이터 전처리
Detection model

민교

회의록 작성
Tracking Metric
결과 분석

손승우

Tracking model
결과 분석

프로젝트 수행 절차 및 방법

- 업무 분담 프로세스
  - ➢분리 가능한 영역으로 나눠 해당 분야를 학습 후 공유
    - 공유 방식
      - ❖ 화 / 금 온라인 회의
    - 공유 내용
      - ❖ 데이터 전처리 동기화
        - 클래스 결합
      - ❖ 구현한 코드, 실행 결과 공유
        - Github merging
    - 학습 내용
      - ❖ 적용할 방법론 학습
        - 고민수: YOLOv3, YOLOv4
        - 김민교: Detecter loss와 metric, Tracker metric
        - 손승우: DeepSORT, ByteTrack

프로젝트 수행 절차 및 방법

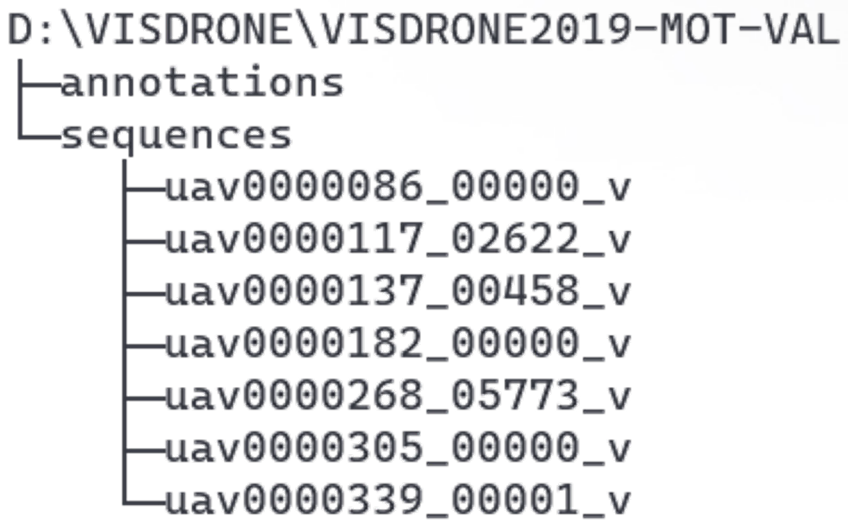
- 수행 절차

구분	기간(2023년)	활동	비고
모델 선정	6/30(금) ~ 7/4(월)	• 베이스 모델 탐색	• YOLOv3, DeepSORT로 선정
데이터 1차 전처리	6/30(금) ~ 7/4(월)	• Annotation 탐색 • Data frame 제작	• Car, People, pedestrian 레이블 만 사용, 나머지 제거
데이터 제너레이터 구현	6/30(금) ~ 7/9(일)	• tfrecord를 통한 제너레이터 구현 • Data frame을 통한 제너레이터 구현	• 2차 전처리 이후 모두 폐기
베이스 모델 구현	7/10(월) ~ 7/24(월)	• YOLOv3 구현 실패 • DeepSORT 구현 • ByteTrack 구현	• Detection 구현이 지속적으로 실패 • Pretrained detection modeld을 통해 Tracking 수행 및 결과 확인
데이터 2차 전처리	7/24(월)	• Class 결합, 재설정	• 클래스를 제거하는 과정에서 프레임이 삭제된 것을 확인 후 재설정
베이스 모델 2차 구현	7/24(월) ~ 7/28(금)	• YOLOv3, YOLOv4 모델 구현	• Data generator, Anchor 문제 해결
실험 후 결과 분석	7/28(금) ~ 8/3(목)	• YOLOv3,4 -DeepSORT, Bytetrack 실험 완료	• 33%의 frame 만을 이용한 실험 진행
추가 모델 구현 후 실험	8/3(목) ~ 8/7(월)	• 다른 실험 조건에서의 실험 완료	-
결과 분석 및 보고서 작성	8/7(월) ~ 8/10(목)	• 모델 별, 비디오 별 실험 분석	-
총 프로젝트 시간	6/30(금) ~ 8/10(목) (총 6주)	-	-

EDA

Visdrone dataset (1/

- 데이터 구성
  - ➢Image by frame, Annotation
    - 하나의 비디오에서 나온 프레임 별 이미지와 어노테이션
  - ➢Train / Valid / Test
    - 비디오의 개수와 크기
      - ❖ Train: 56개 (7.53 GB)
      - ❖ Valid: 7개 (1.48 GB)
      - ❖ Test: 17개 (2.14 GB)
    - 구성
      - ❖ Sequences
        - 각 video의 프레임 별 이미지가 담긴 폴더
        - 각 프레임은 0000001.jpg 와 같은 형태
      - ❖ Annotations
        - video의 Annotation



Visdrone dataset의 valid dataset 폴더의 tree

Visdrone dataset (2/

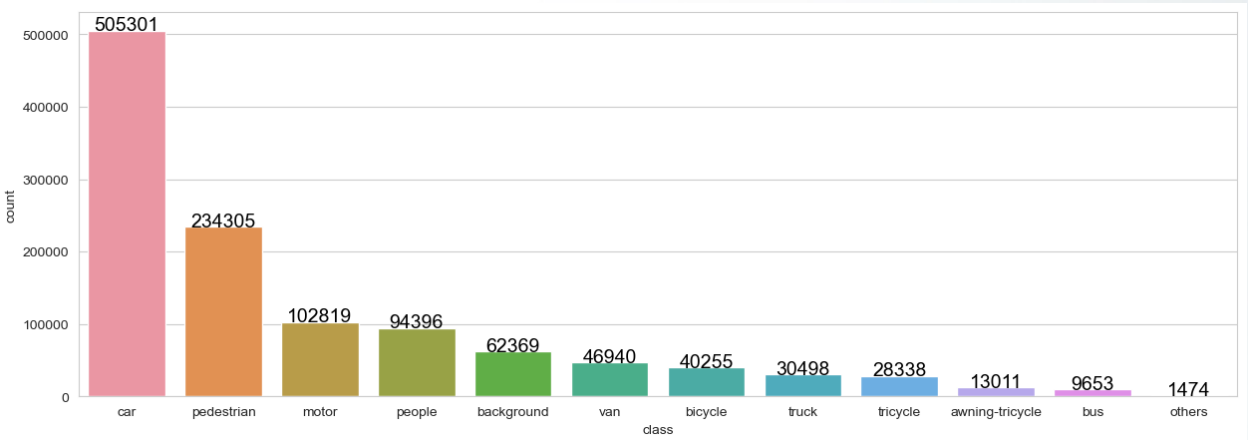
- Annotation 탐색

frame_index	target_id	bbox_x	bbox_y	bbox_width	bbox_height	score	object_ category	truncation	occlusion
98	0	808	1	47	22	1	4	0	0
99	0	808	55	46	21	1	4	0	0
100	0	806	112	47	23	1	4	0	0
101	0	802	122	47	22	1	4	0	0
102	0	800	122	47	22	1	4	0	0
103	0	799	123	47	22	1	4	0	0

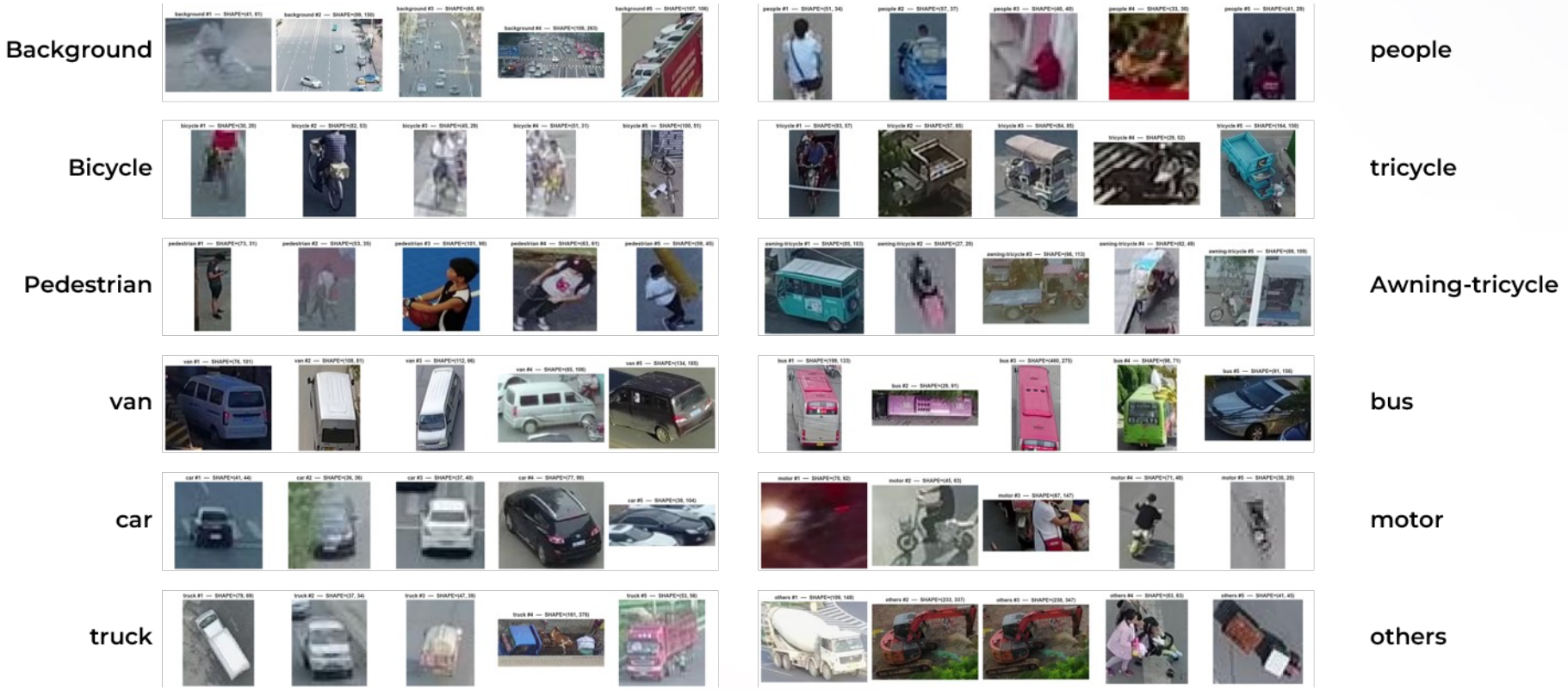
1. frame_index : **비디오 프레임의 프레임 인덱스**
2. target_id : **다른** frame **과의 시간적 대응 관계를 제공하는 데 사용**
3. bbox_x : bbox**의** left top**의** x **좌표**
4. bbox_y : bbox**의** left top**의** y **좌표**
5. bbox_width : bbox **의 너비**
6. bbox_height : bbox **의 높이**
7. score : bbox**의 신뢰도** (1, 0)
8. object_category : **클래스 번호** (0~11)
9. truncation : **객체가 이미지 프레임에 의해 잘려진 정도** (0: **없음**, 1: **부분적 잘림**)
10. occlusion : **객체가 가려진 정도** (0: 0%, 1: 1~50%, 2: 50~100%)

EDA (1/7)

- Samples by Category

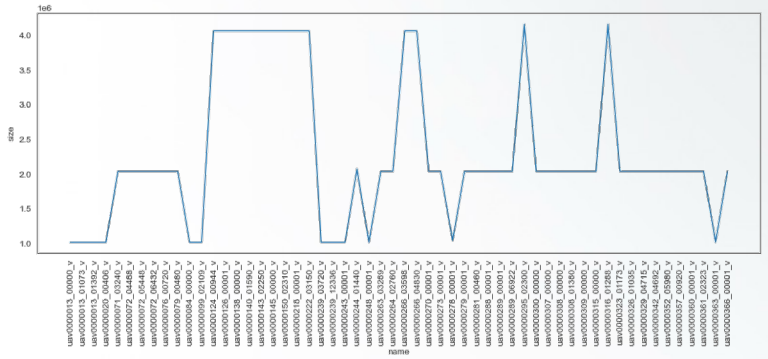


Visdrone train dataset의 클래스 별 객체 개수

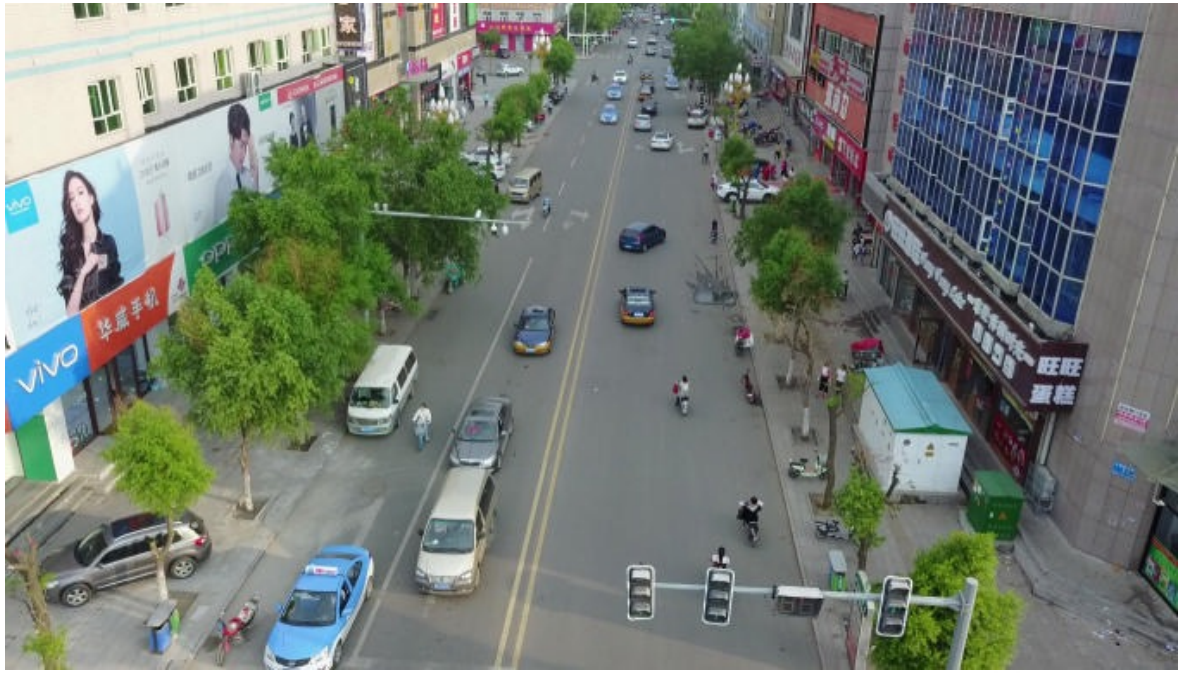


EDA (2/7)

- 비디오 별 데이터 셋 탐색
  - ➢이미지 크기의 차이
    - 좌측: 682 ×382 크기의 이미지
    - 우측: 2688×1512 크기의 이미지



Visdrone train dataset의 비디오 별 이미지 크기 plot



EDA (3/7)

- 비디오 별 데이터 셋 탐색
  - ➢낮과 밤의 차이
    - Train dataset: 낮 (52) / 밤 (4)
    - Valid dataset: 낮 (1) / 밤 (6)
    - Test dataset: 낮 (13) / 밤(4)

EDA (4/7)

- 비디오 별 데이터 셋 탐색
  - ➢촬영 각도의 차이

EDA (5/7)

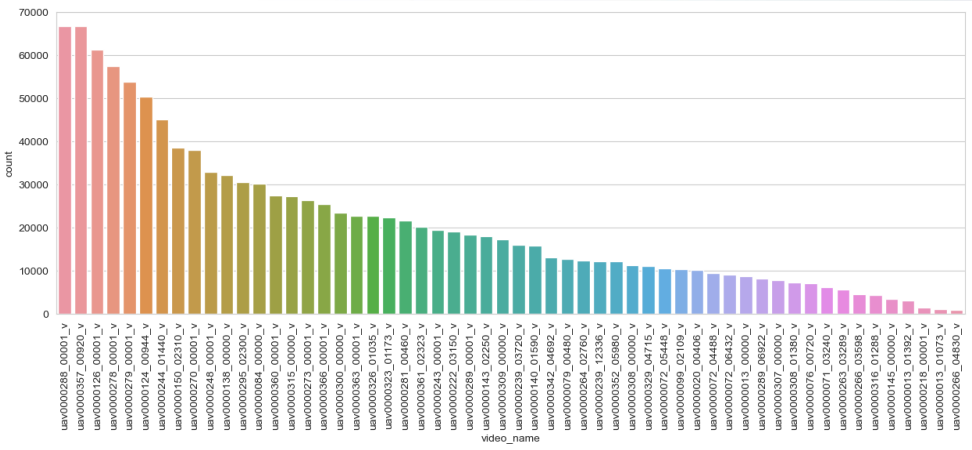
- 비디오 별 데이터 셋 탐색
  - ➢클래스 불균형

EDA (6/7)

- 비디오 별 데이터 셋 탐색
  - ➢Blurry한 이미지

EDA (7/7)

- 비디오 별 데이터 셋 탐색
  - ➢Crowd 정도

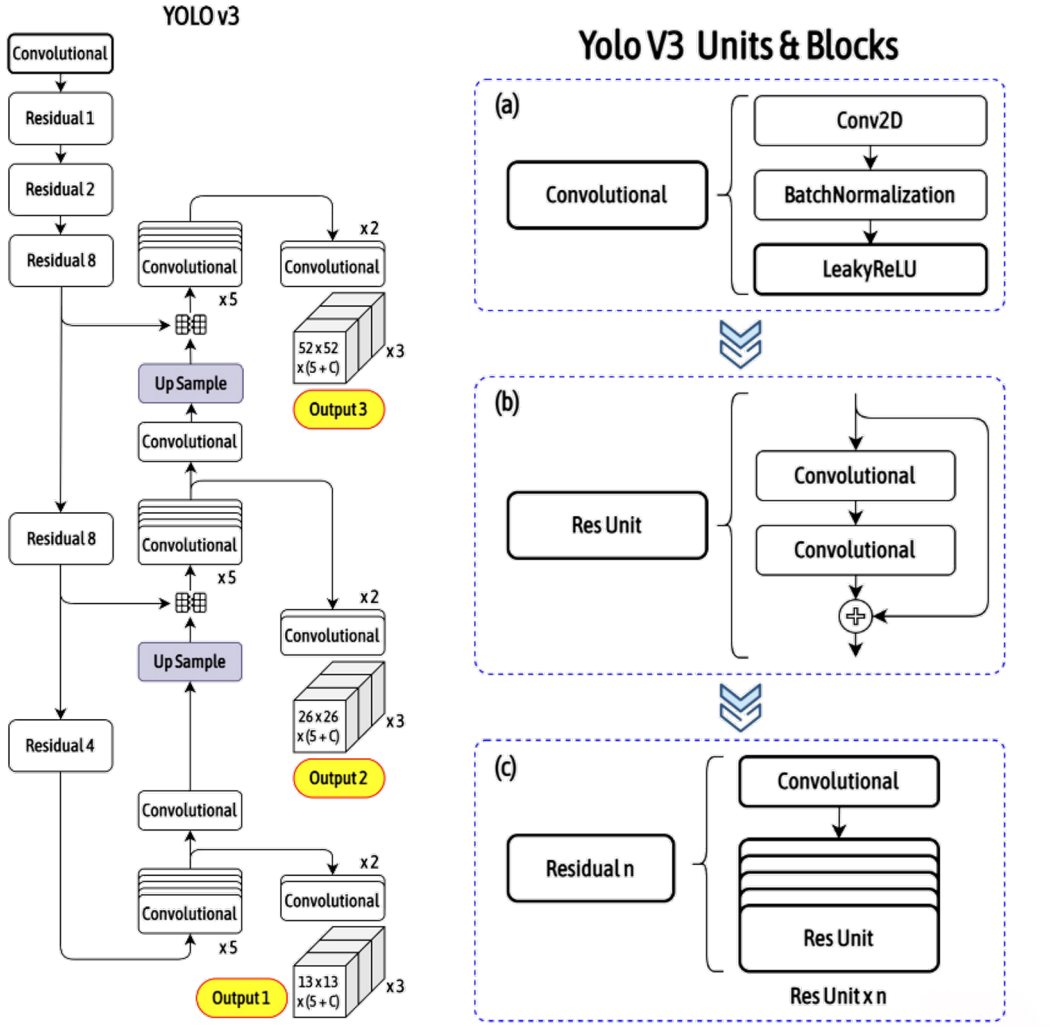


Visdrone train dataset의 비디오 별 객체 개수

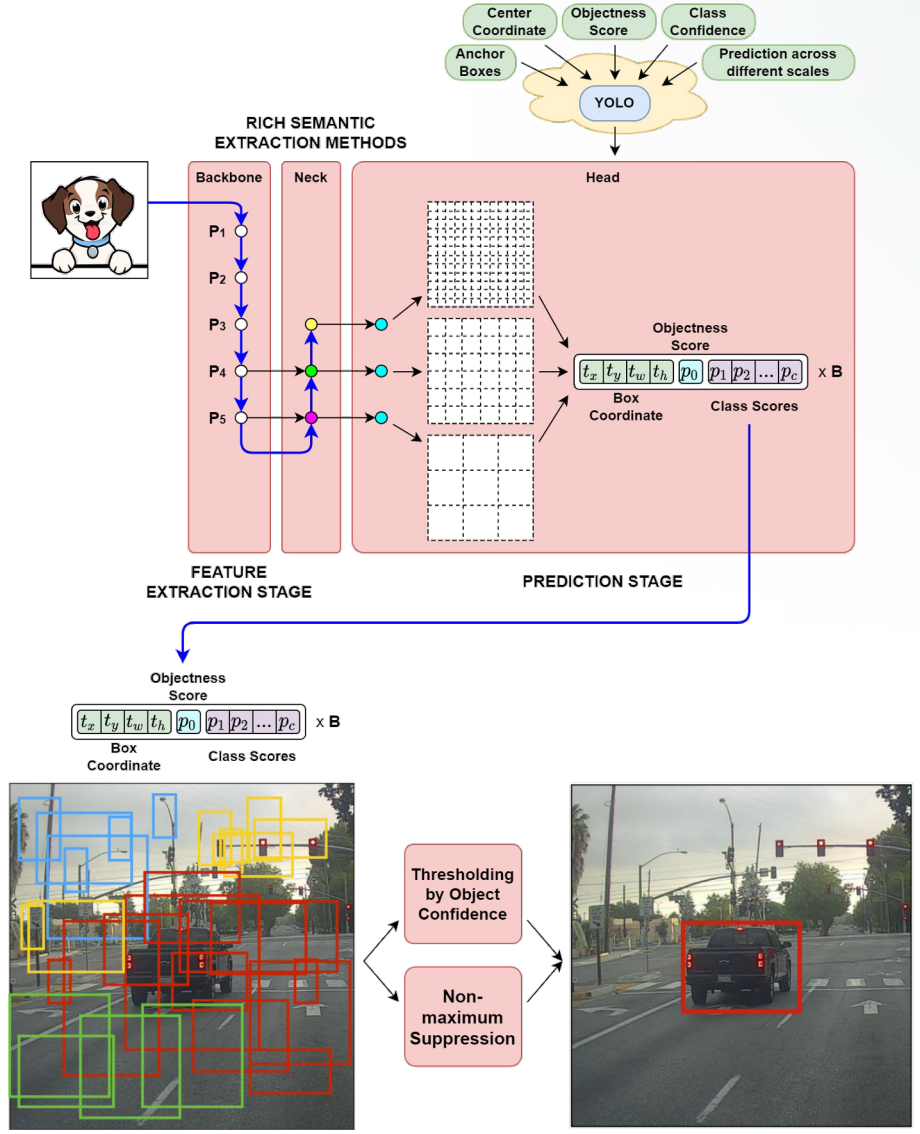
방법론

Detection Baseline model, YOLO

- Architecture & Workflow



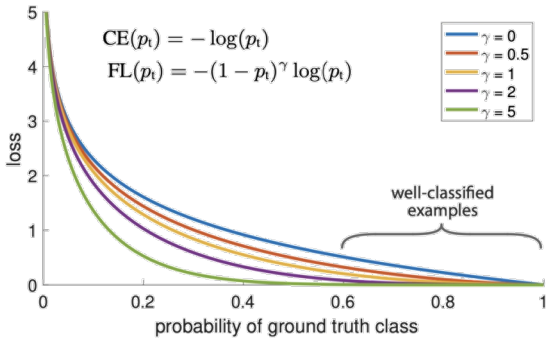
YOLOv3의 Architecture



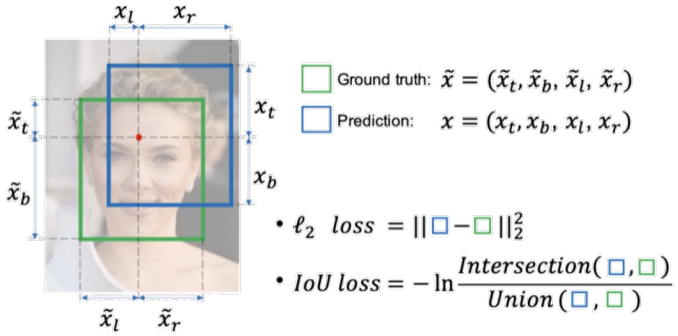
32. Yolo V4 explained in full detail-EN - Deep Learning Bible - 4. Object Detection - Eng. (wikidocs.net)

Detection Baseline model, YOLO

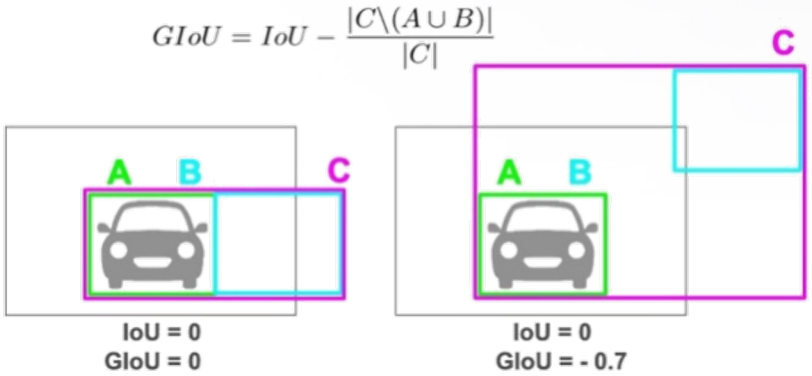
- Loss
  - ➢Focal Loss
    - 훈련 과정에서 background와 클래스 사이에 극심한 불균형 있을 때 조정하는 Loss
  - ➢GIoU Loss
    - 기존의 IoU Loss가 겹치지 않을 때 학습을 제공하지 않는 문제점을 개선



γ값 에 따른 Focal Loss와 probability



IoU Loss 설명



GIoU Loss 설명

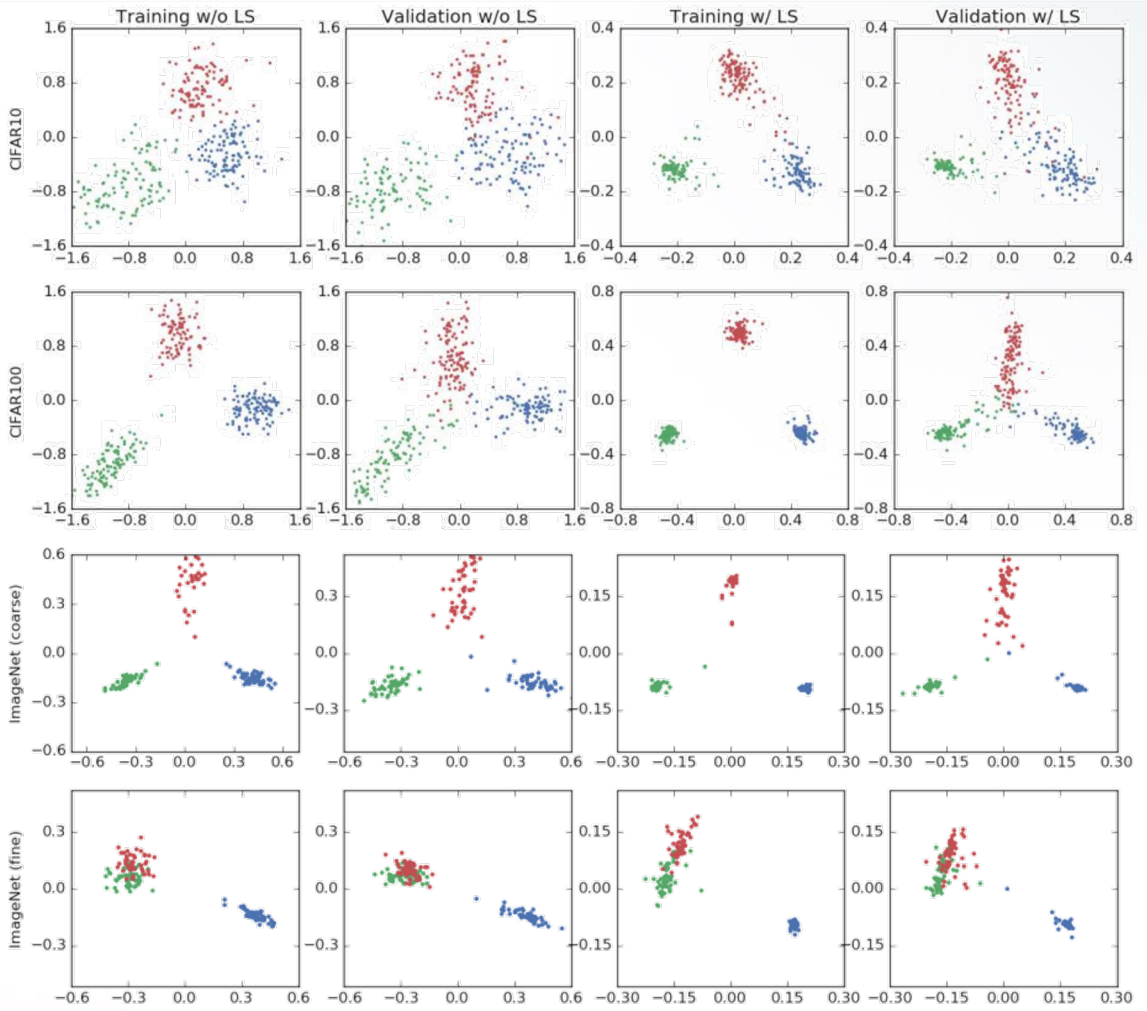
```
conf_focal = tf.pow(respond_bbox - pred_conf, 2)
conf_loss = conf_focal * (
        respond_bbox * tf.nn.sigmoid_cross_entropy_with_logits(labels=respond_bbox, logits=conv_raw_conf)
        +
        respond_bgd * tf.nn.sigmoid_cross_entropy_with_logits(labels=respond_bbox, logits=conv_raw_conf)
)
```

사용한 Focal Loss의 code

Detection Baseline model, YOLO

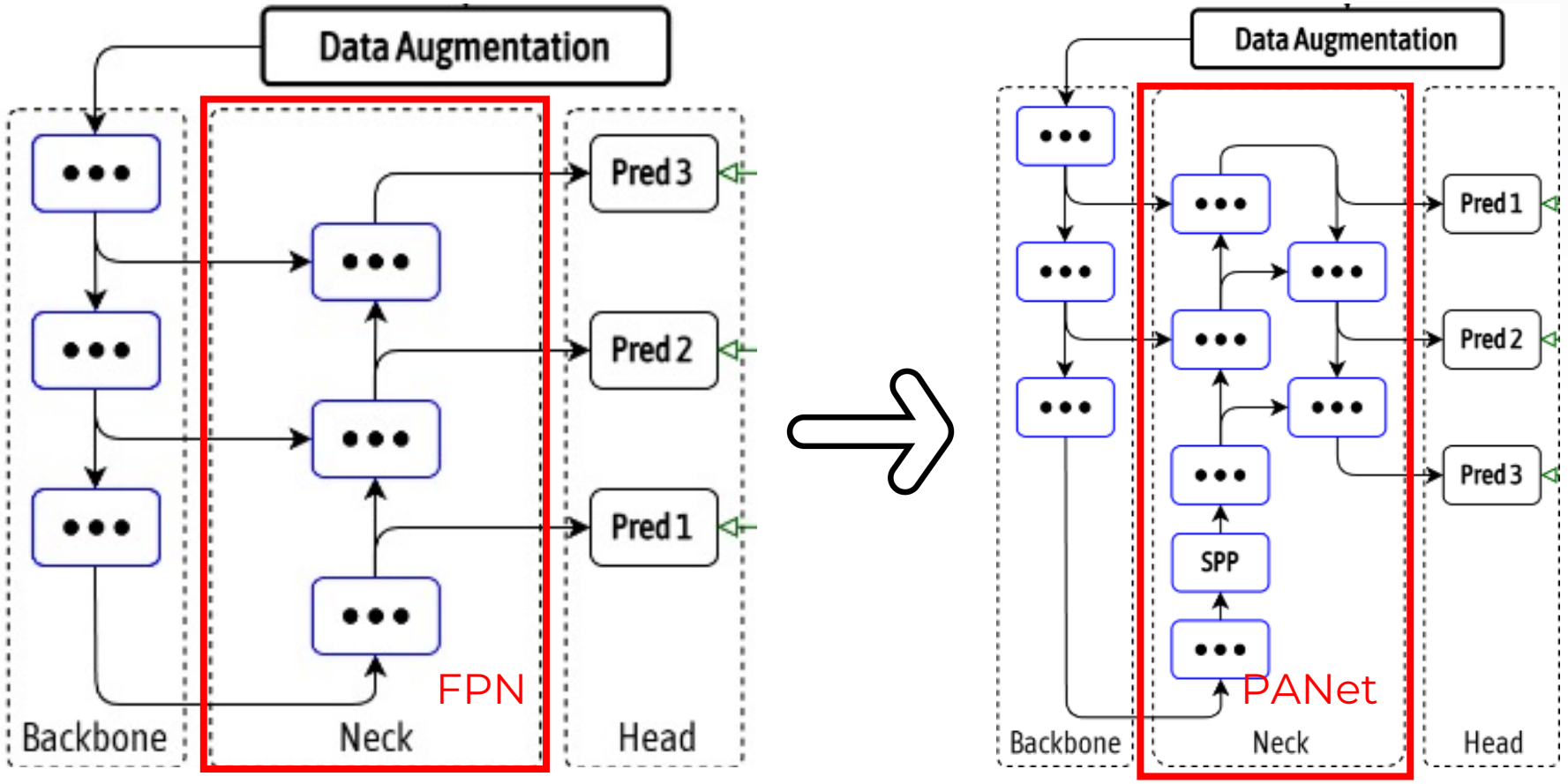
- Label Smoothing

```
tf.losses.softmax_cross_entropy(
    onehot_labels,
    logits,
    weights=1.0,
    label_smoothing=0,
    scope=None,
    loss_collection=tf.GraphKeys.LOSSES,
    reduction=Reduction.SUM_BY_NONZERO_WEIGHTS
)
```



YOLO Improvements

- Neck module 변경

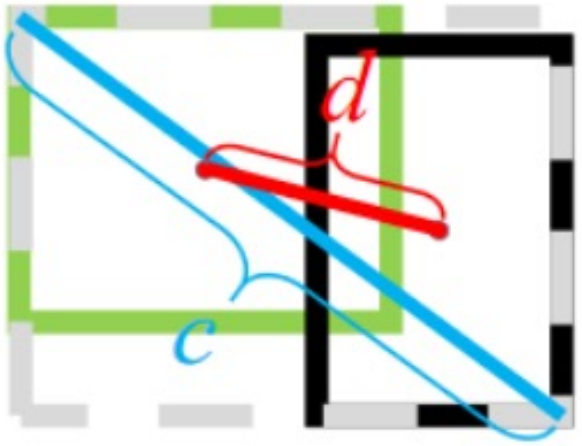


YOLOv3의 workflow

YOLO Improvements

- Bounding Box Loss 변경
  - ➢DIoU, CIoU
    - DIoU: 박스의 중심점을 강제로 가까이 하도록 Loss를 설정
    - CIoU: DIoU에서 박스의 종횡비의 일관성을 유지하도록 Loss를 설정

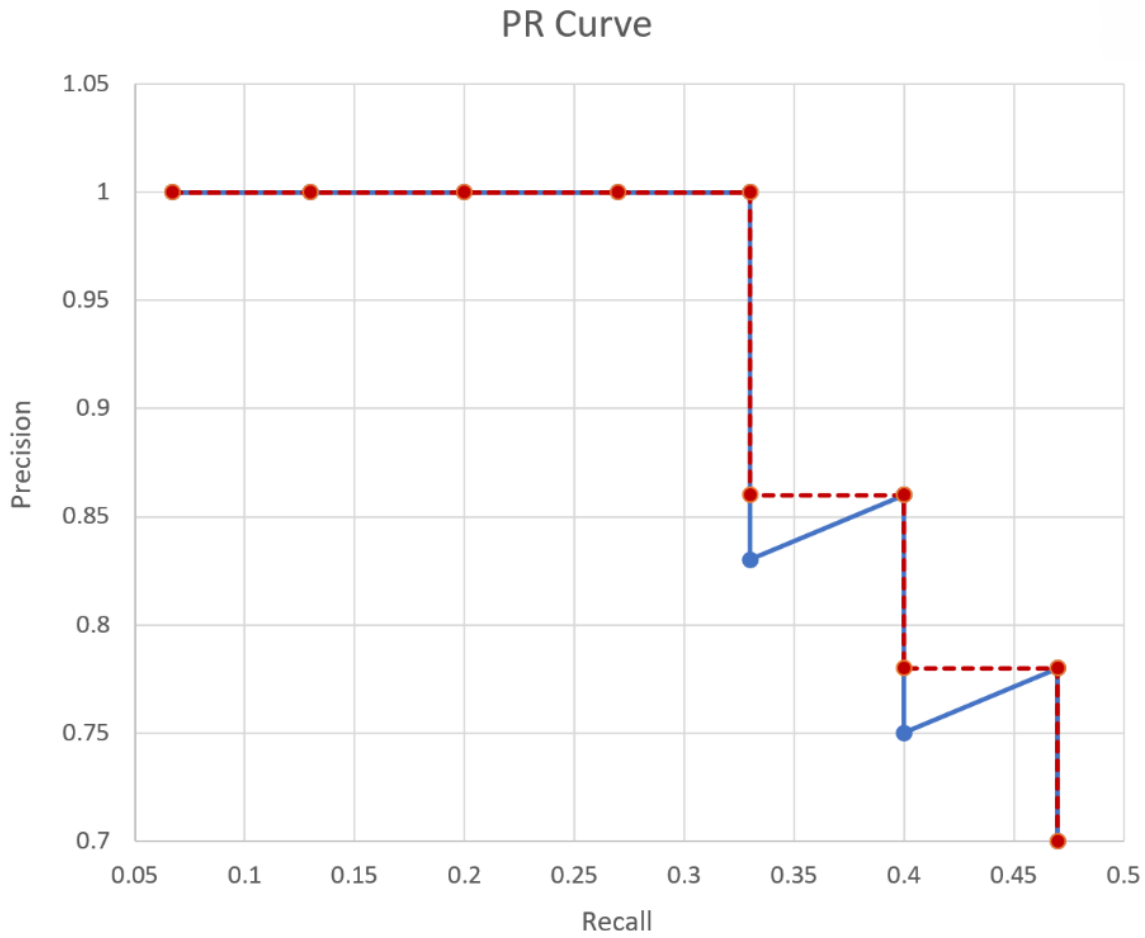
$$\mathcal{L}_{DIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2}$$



DIoU에서의 두 박스

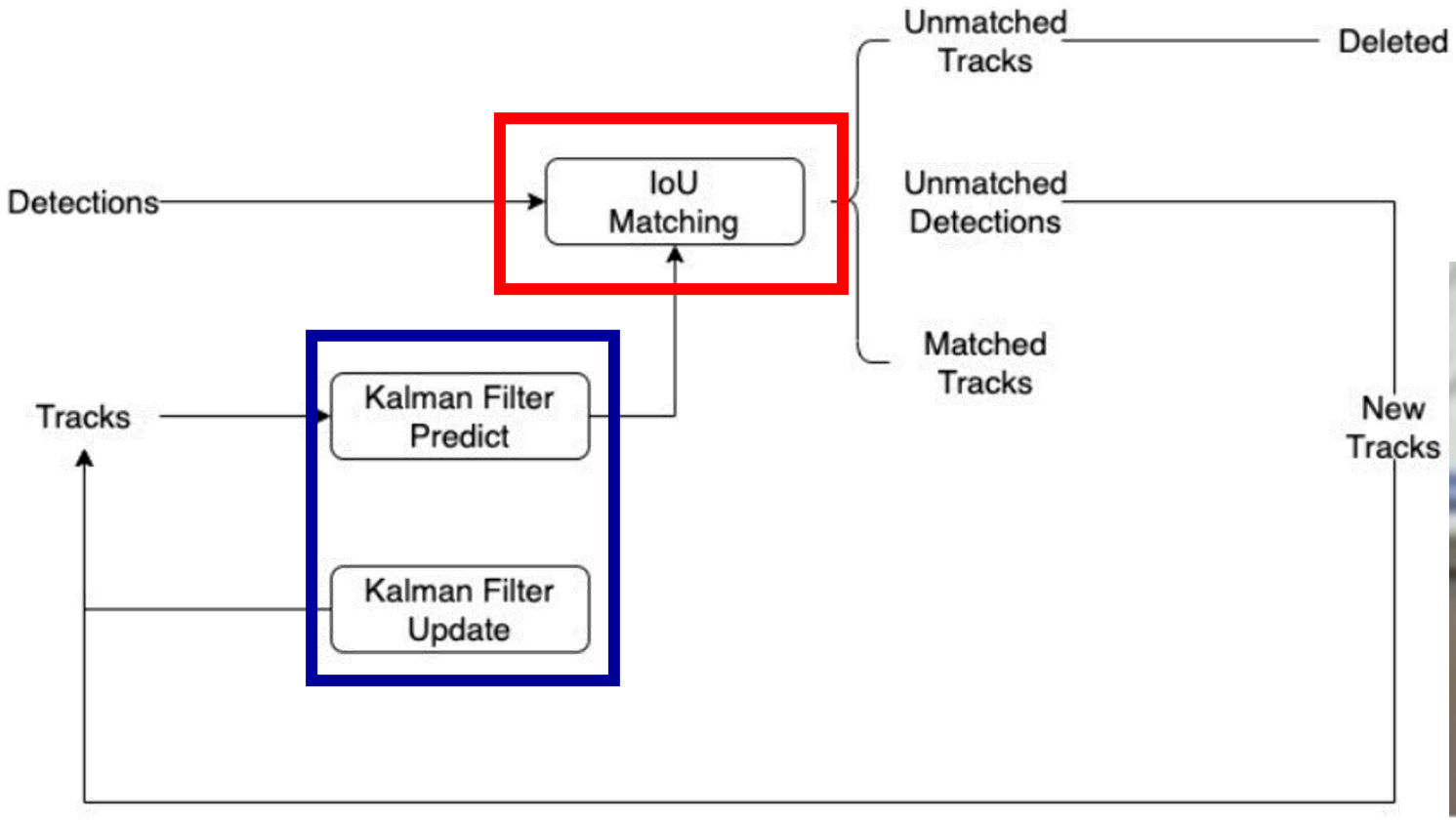
YOLO Metric

- mAP(IoU50%)
  - ➢Precision-Recall Curve에서 단조적으로 감소하는 그래프로 변환 후 아래의 면적

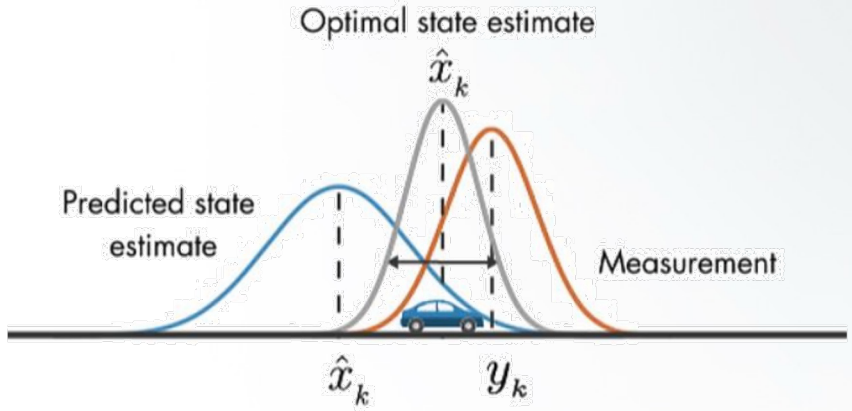


SORT

- Architecture & Workflow



SORT의 Workflow

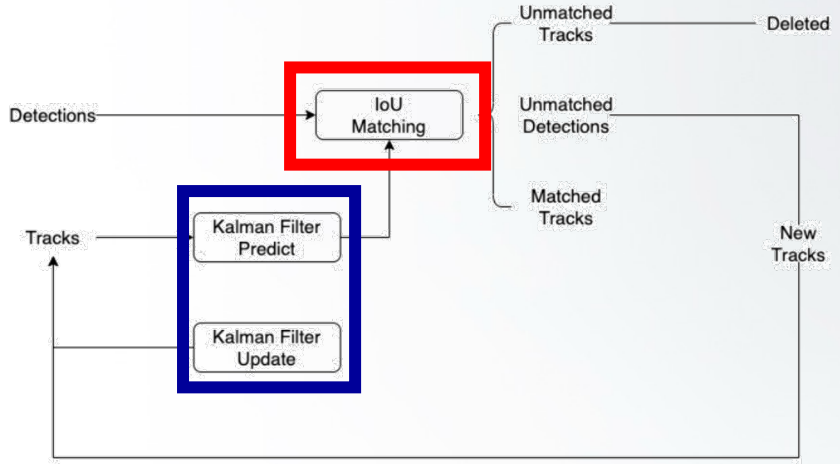


Kalman filter의 설명

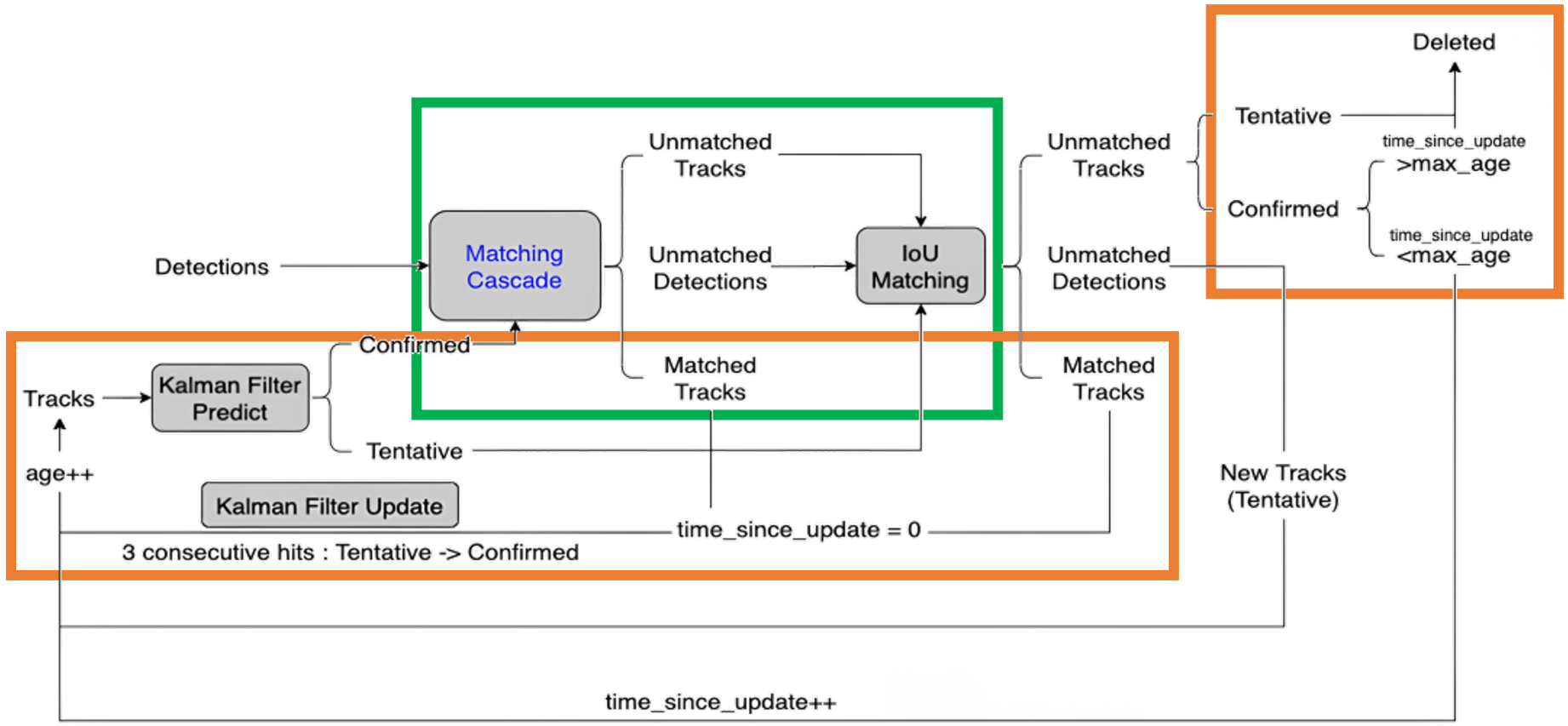
Kalman filter가 가지는 값의 예시

DeepSORT

- Architecture & Workflow



SORT의 Workflow



ByteTrack

- Architecture & Workflow

Algorithm 1: Pseudo-code of BYTE.

Input: A video sequence V; object detector Det; detection score threshold τ

Output: Tracks $\mathcal{T}$ of the video

Initialization: $\mathcal{T} \leftarrow \emptyset$

for *frame* f_k *in* V **do**

  /* Figure 2(a) */

  /* predict detection boxes & scores */

  $\mathcal{D}_k \leftarrow \text{Det}(f_k)$

  $\mathcal{D}_{high} \leftarrow \emptyset$

  $\mathcal{D}_{low} \leftarrow \emptyset$

  for d *in* $\mathcal{D}_k$ **do**

    if $d.score > \tau$ **then**

      $\mathcal{D}_{high} \leftarrow \mathcal{D}_{high} \cup \{d\}$

    end

    else

      $\mathcal{D}_{low} \leftarrow \mathcal{D}_{low} \cup \{d\}$

    end

  end

  /* predict new locations of tracks */

  for t *in* $\mathcal{T}$ **do**

    $t \leftarrow \text{KalmanFilter}(t)$

  end

  /* Figure 2(b) */

  /* first association */

  Associate $\mathcal{T}$ and $\mathcal{D}_{high}$ using Similarity#1

  $\mathcal{D}_{remain} \leftarrow$ remaining object boxes from $\mathcal{D}_{high}$

  $\mathcal{T}_{remain} \leftarrow$ remaining tracks from $\mathcal{T}$

  /* Figure 2(c) */

  /* second association */

  Associate $\mathcal{T}_{remain}$ and $\mathcal{D}_{low}$ using similarity#2

  $\mathcal{T}_{re-remain} \leftarrow$ remaining tracks from $\mathcal{T}_{remain}$

  /* delete unmatched tracks */

  $\mathcal{T} \leftarrow \mathcal{T} \setminus \mathcal{T}_{re-remain}$

  /* initialize new tracks */

  for d *in* $\mathcal{D}_{remain}$ **do**

    $\mathcal{T} \leftarrow \mathcal{T} \cup \{d\}$

  end

end

Return: $\mathcal{T}$

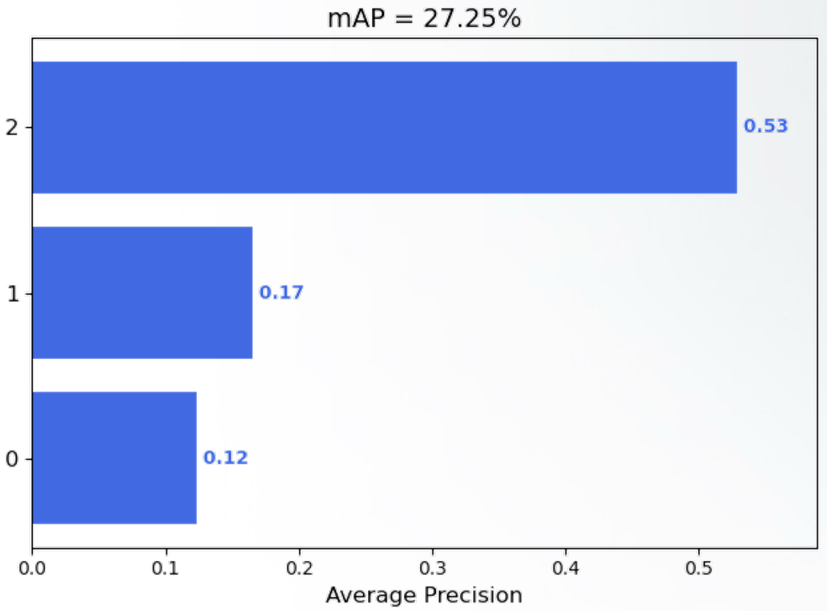
Tracking

- MOTA(MOT-Accuracy)
  - ➢비디오나 시퀀스에서 여러 개의 객체를 추적하는 능력을 평가하는 지표
    - $MOTA = 1 - (FN + FP + IDSW)/GT$
      - ❖ FN: 실제 객체를 놓친 수
      - ❖ FP: 잘못된 객체로 판단한 수
      - ❖ $IDSW$: 객체의 ID가 바뀐 경우의 수
      - ❖ GT: 실제 총 객체의 수
- IDF1(IDentity F1 Score)
  - ➢객체의 식별자(ID)를 정확하게 추적하는 능력을 평가하는 지표
    - $1DF1 = 2 * (IDP \times IDR)/(IDP + IDR)$
      - ❖ IDP: Identity precision으로 추적한 객체 중 올바르게 추적한 ID를 가진 비율
      - ❖ IDR: Identity Recall 실제 객체 중 올바른 식별자를 가진 객체의 비율

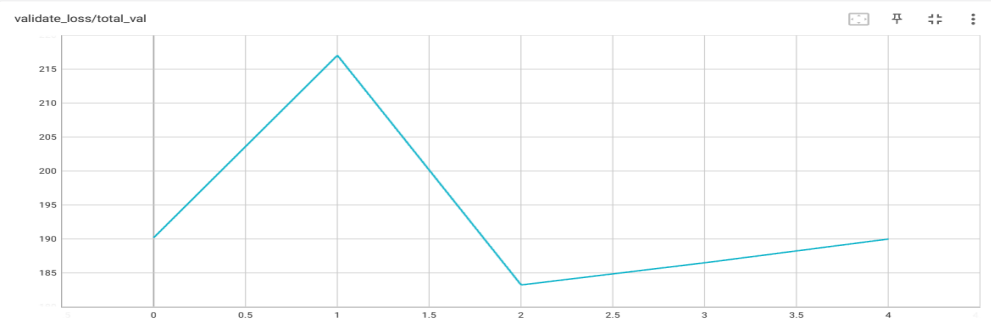
실험

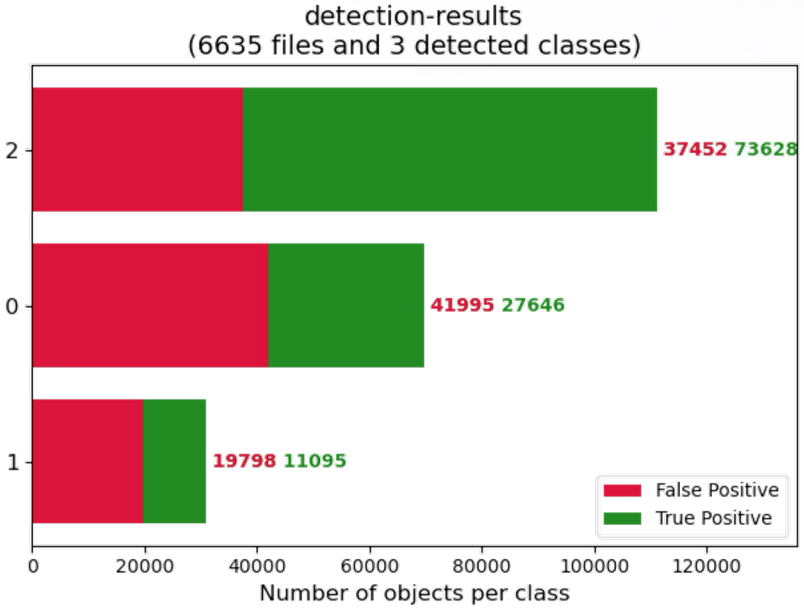
Detection

Class 0 : People
Class 1 : Motor
Class 2 : Car



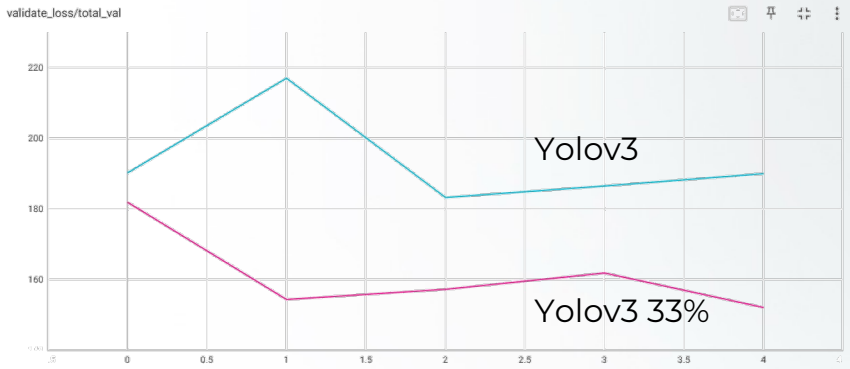
- Baseline YOLOv3
  - ➢Loss: Focal Loss + GIoU Loss + Prob Loss
  - ➢Label smoothing
  - ➢Locational augmentation
  - ➢Input image size: 416×416
  - ➢Max anchor box: 100
  - ➢Anchor per scale: 3
  - ➢Train with Full trainset



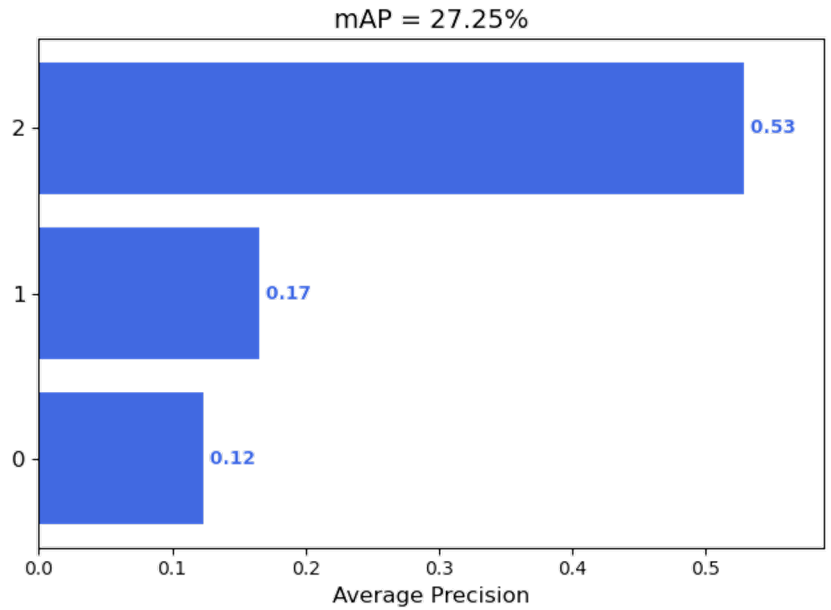


Detection

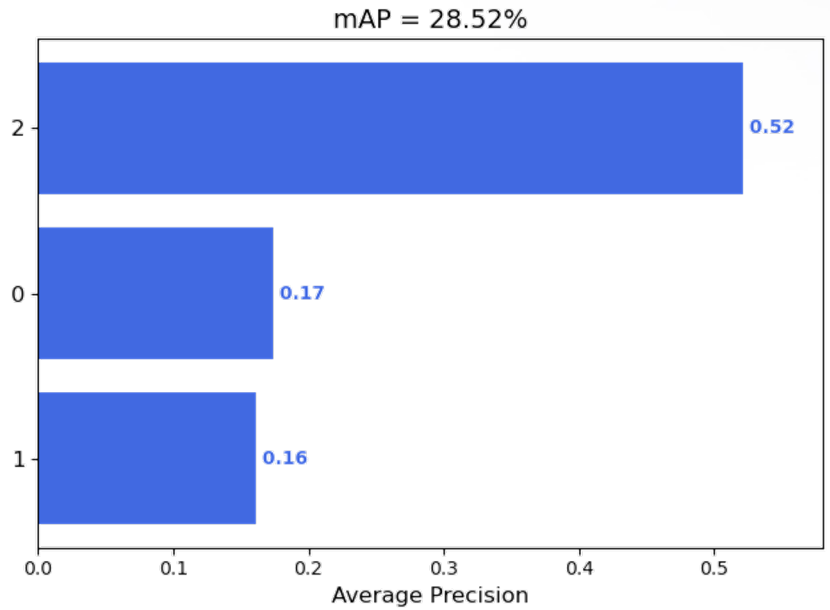
- YOLOv3 33%
  - ➢전체 Trainset의 33%, 3번에 1번씩의 프레임을 사용
    - 성능 면에서 큰 차이가 없어 제한적인 시간을 이유로 33% 데이터를 이용하여 실험을 진행



5 에폭에서의 valid total loss 비교



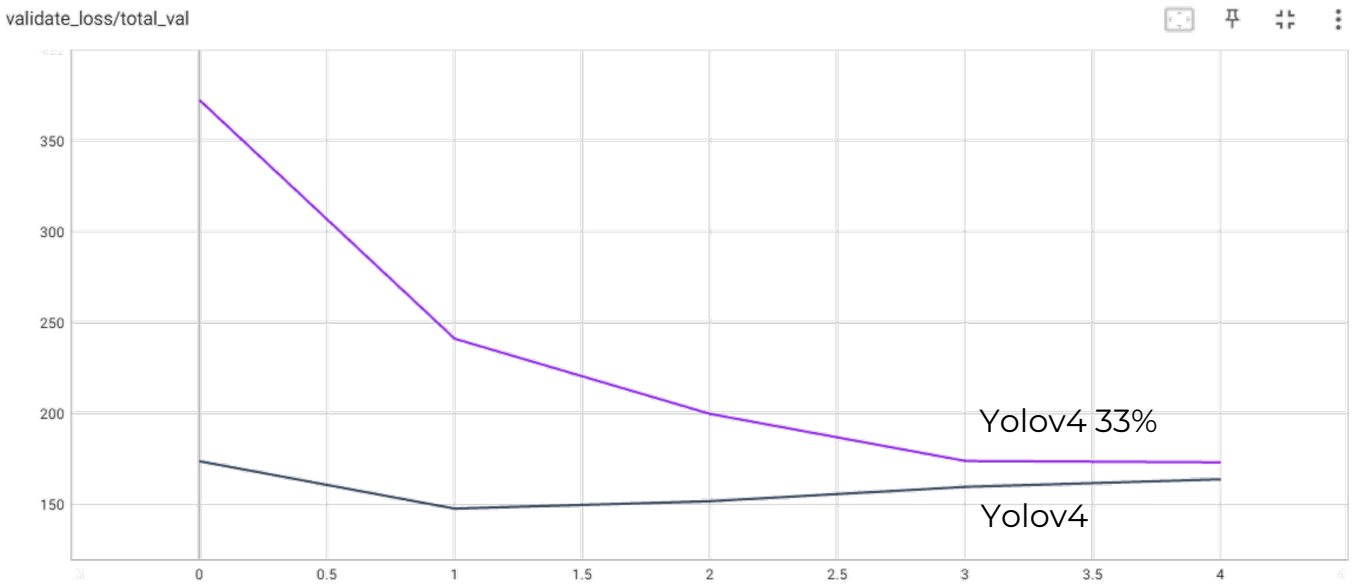
YOLOv3 mAP



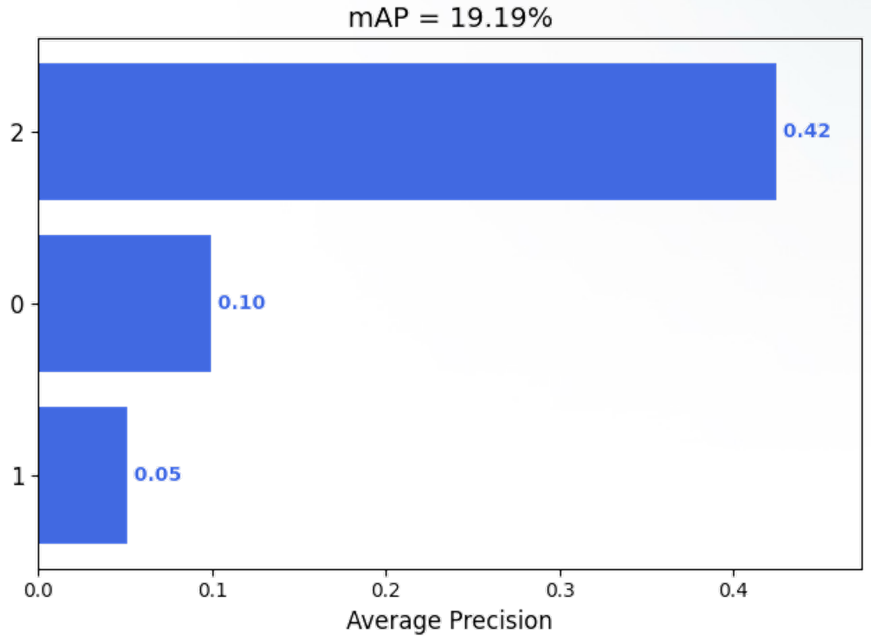
YOLOv3 33% mAP

Detection

- YOLOv4, YOLOv4 33%
  - ➢YOLOv3에 비해 성능이 더 떨이진 모습을 보임



5 에폭에서의 valid total loss 비교

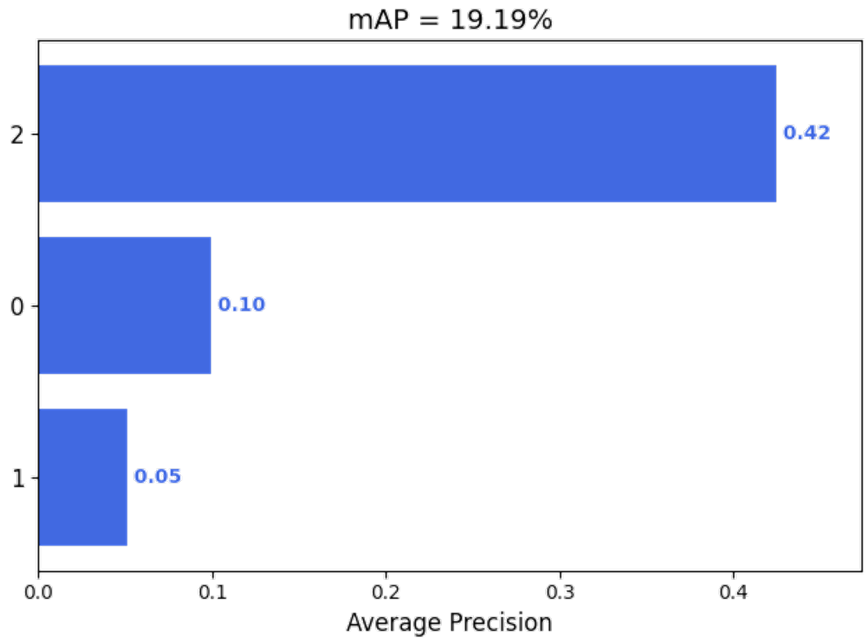


YOLOv4 33%의 mAP

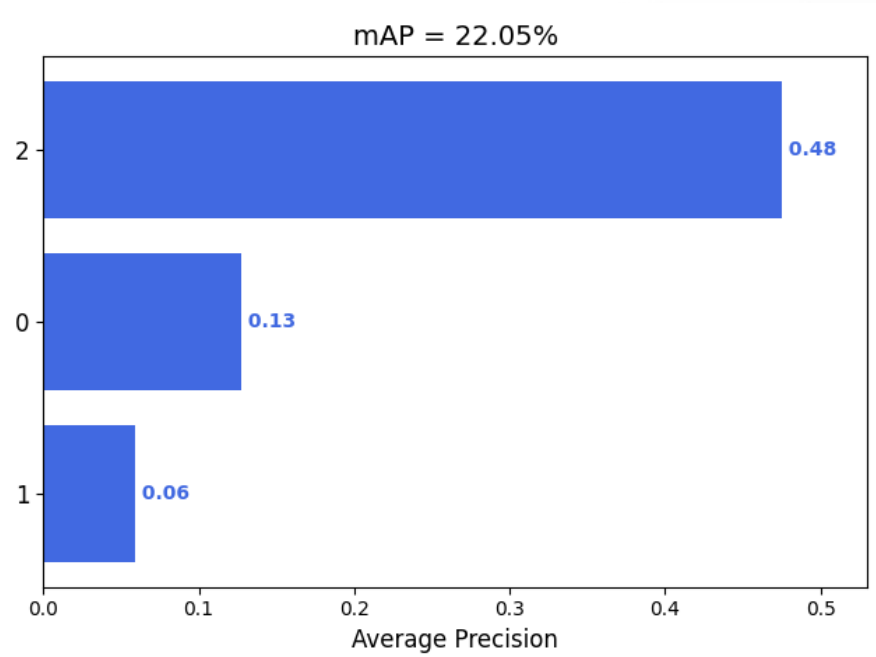
Detection

- YOLOv4 33%, YOLOv4 33% Augmentation
  - ➢Colorjitter, Blur, CLAHE aumgentation을 추가로 사용했을 때 더 높은 mAP

```
A.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4, hue=0.4, p=0.5),
A.Blur(p=0.1),
A.CLAHE(p=0.1),
```



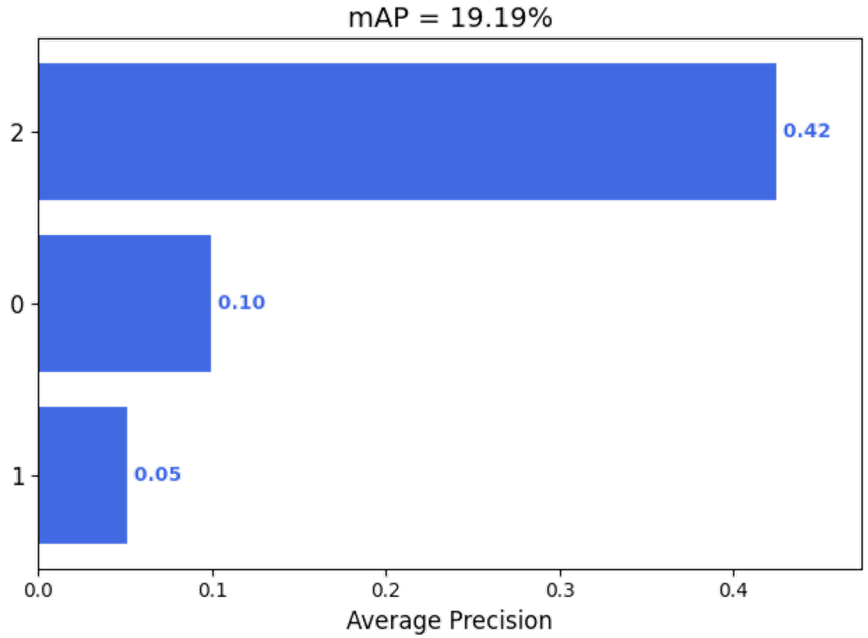
YOLOv4 33%의 mAP



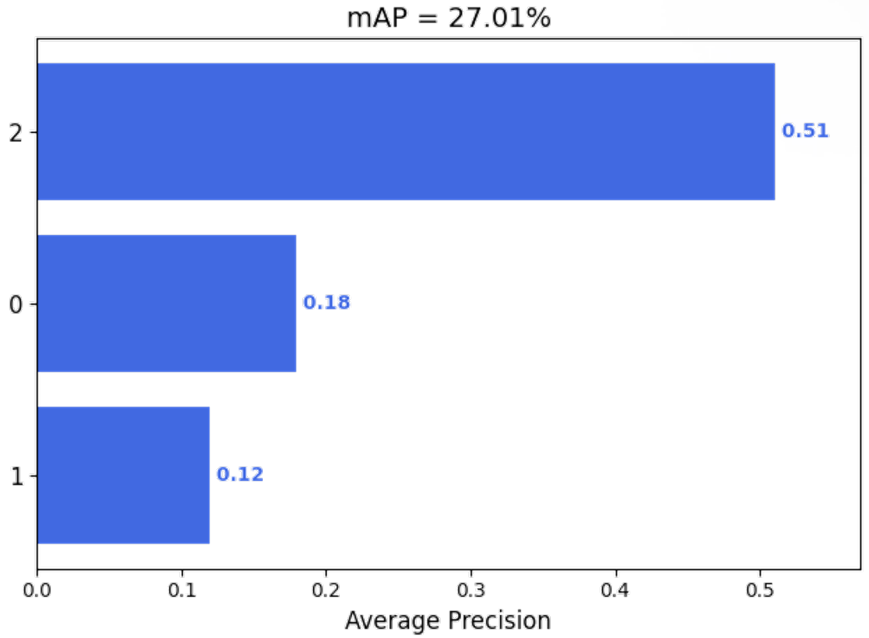
YOLOv4 33% Aumgentaton의 mAP

Detection

- YOLOv4 33%, YOLOv4 33% Max Anchor 150
  - ➢Max Anchor를 100에서 150으로 올렸을 때 더 높은 점수
    - Crowd가 심한 이미지에서 효과가 있을 것으로 예측
    - 200으로 더 높여서 실험을 진행해볼 필요가 있음



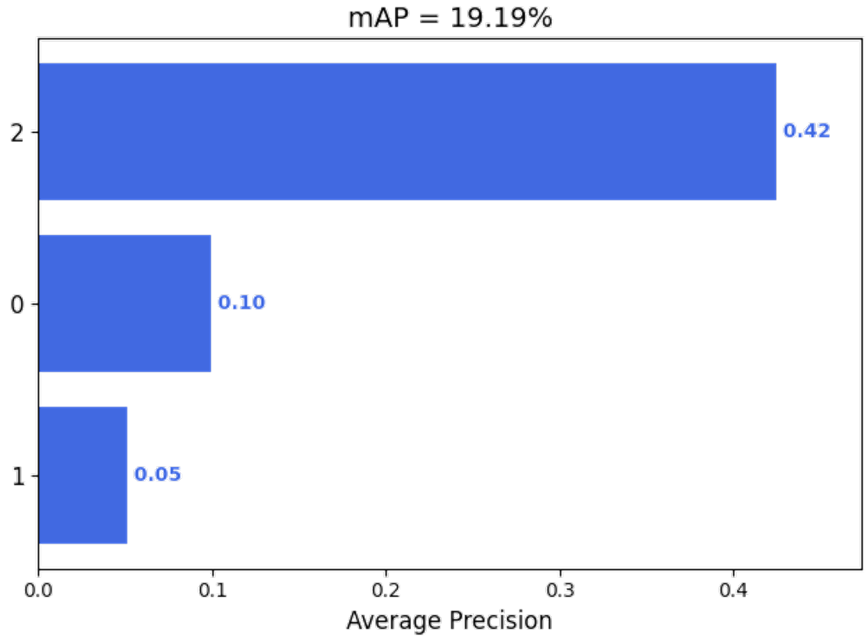
YOLOv4 33%의 mAP



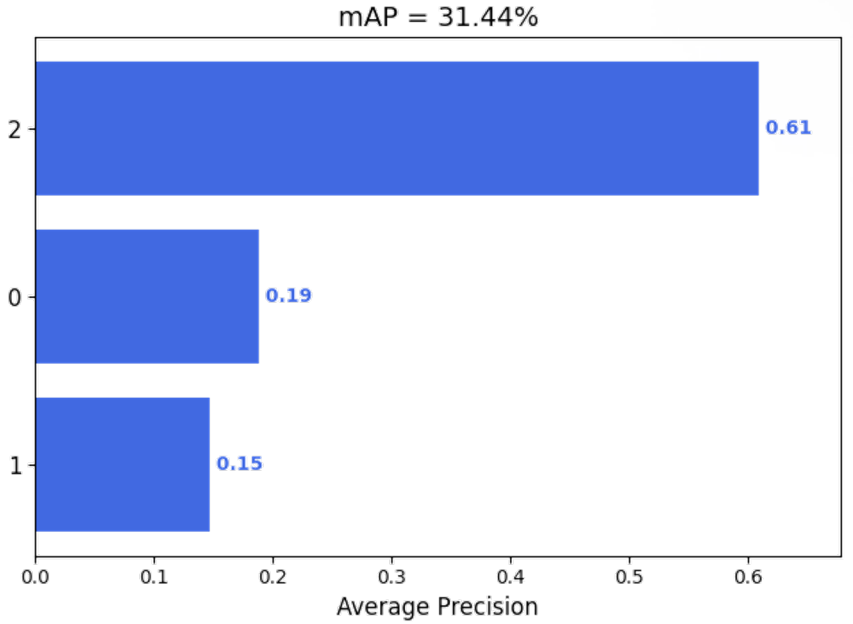
YOLOv4 33% Max anchor 150의 mAP

Detection

- YOLOv4 33%, YOLOv4 33% input size 608
  - ➢Input size를 416에서 608으로 올렸을 때 더 높은 점수
    - 가장 극적인 변화
    - 작은 객체가 대다수인 만큼 큰 이미지에서 오는 이점이 클 것으로 예상



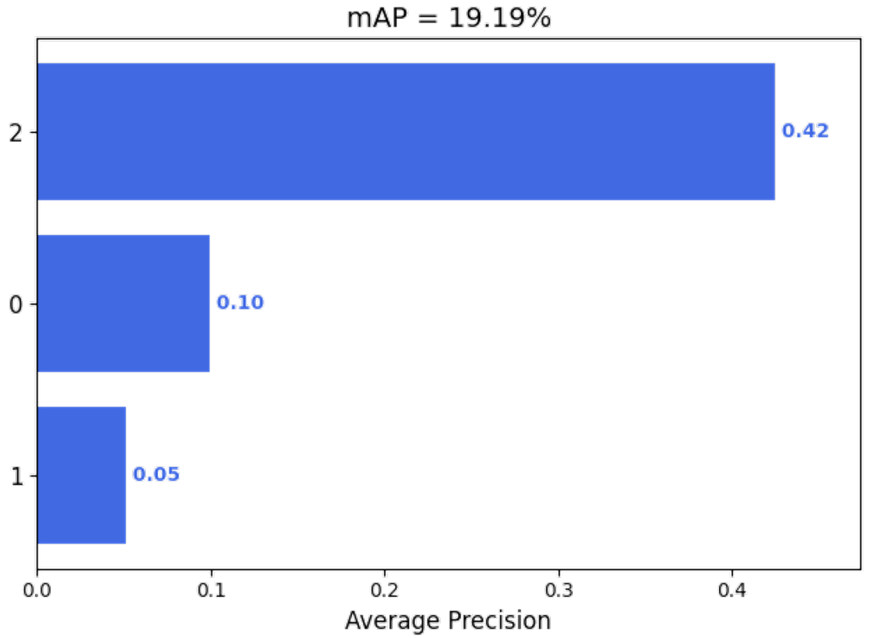
YOLOv4 33%의 mAP



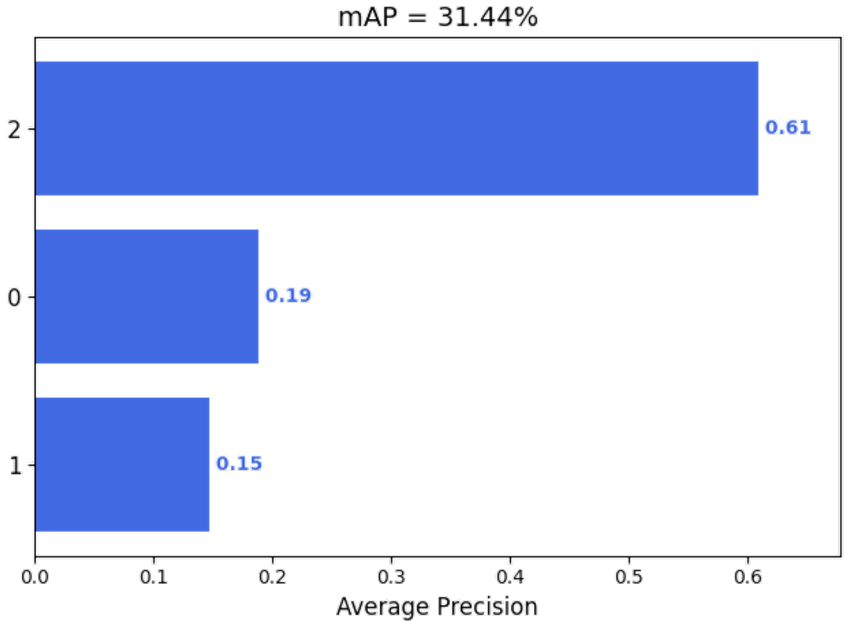
YOLOv4 33% input size 608의 mAP

Detection

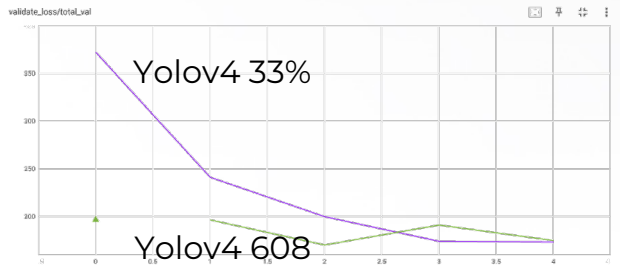
- YOLOv4 33%, YOLOv4 33% input size 608
  - ➢Input size를 416에서 608으로 올렸을 때 더 높은 점수
    - 가장 극적인 변화
    - 작은 객체가 대다수인 만큼 큰 이미지에서 오는 이점이 클 것으로 예상



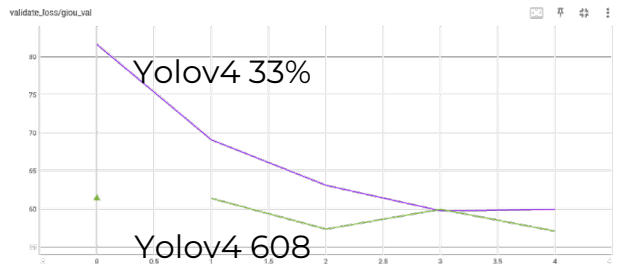
YOLOv4 33%의 mAP



YOLOv4 33% input size 608의 mAP



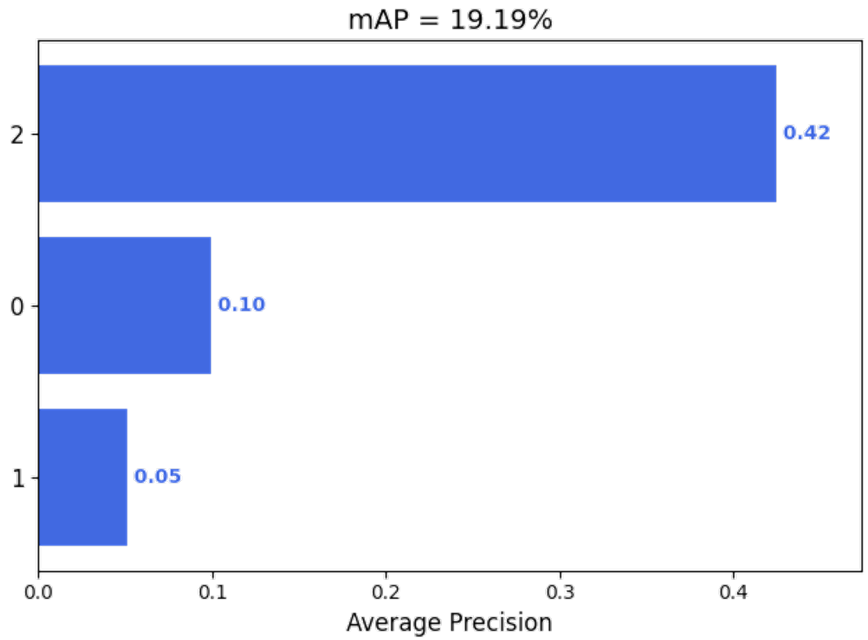
Valid total loss



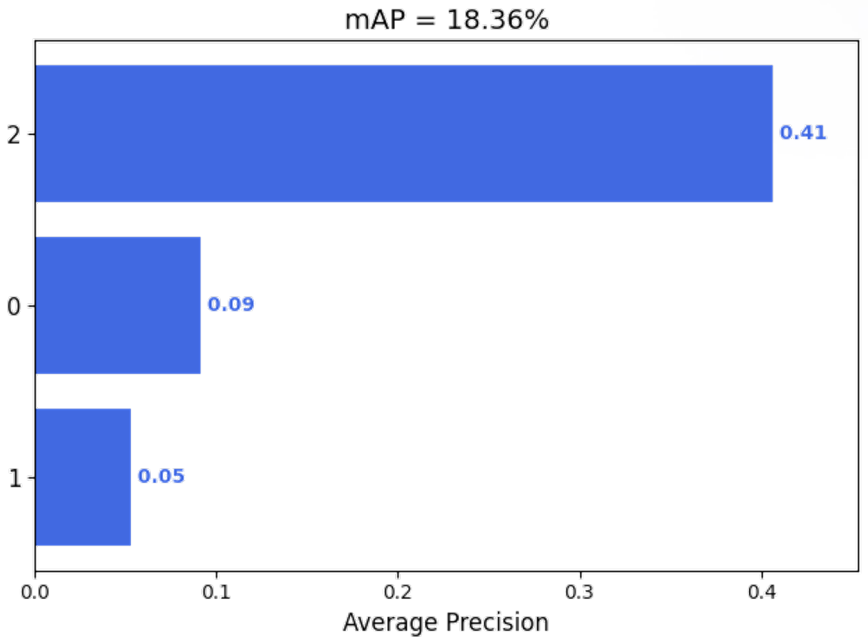
Valid giou loss

Detection

- YOLOv4 33%, YOLOv4 33% No Focal
  - ➢Focal Loss를 사용하지 않았을 때 mAP 감소
    - YOLOv4 33%에서 이미 감지하지 못한 객체가 많아 비교가 힘들



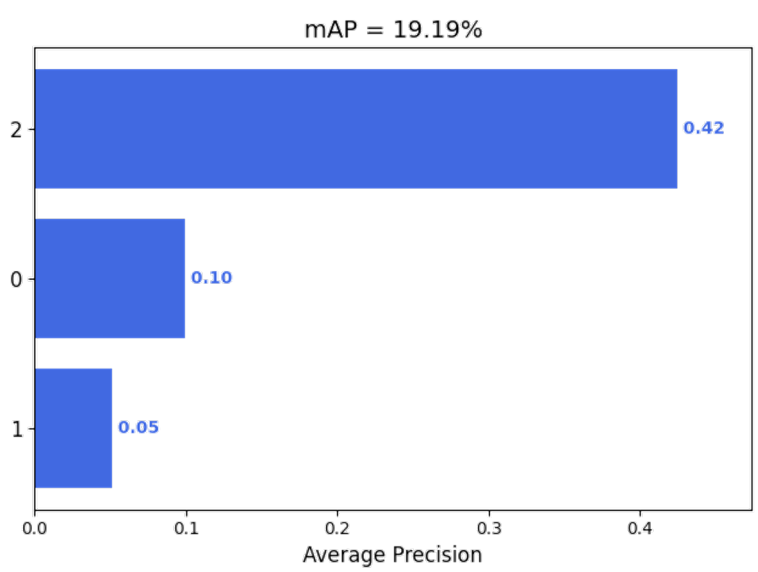
YOLOv4 33%의 mAP



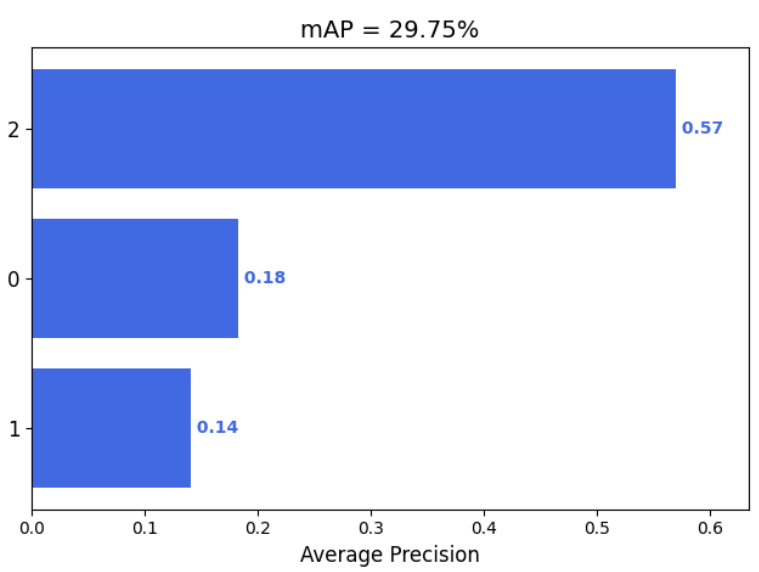
YOLOv4 33% No Focal의 mAP

Detection

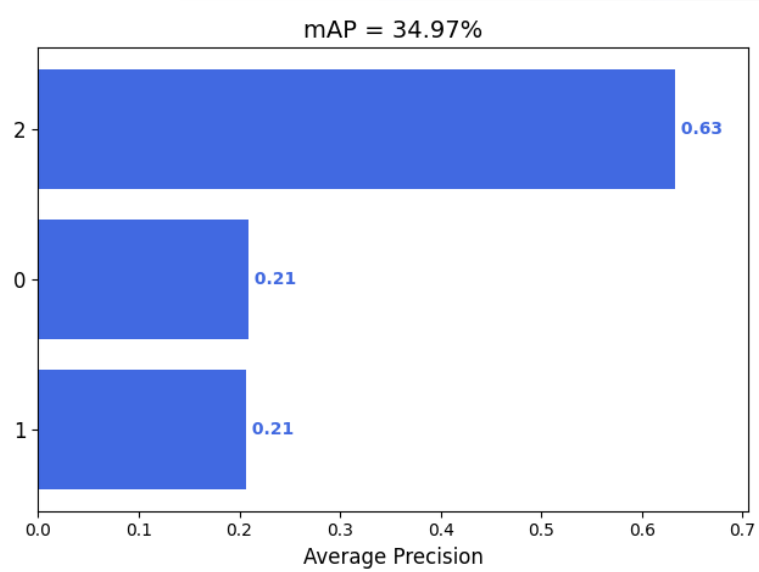
- YOLOv4 33%, YOLOv4 all 33%, YOLOv4 all
  - ➢모든 기법을 다 적용했을 때는 모든 데이터를 다 쓴것과 차이가 크게 나타남



YOLOv4 33%의 mAP



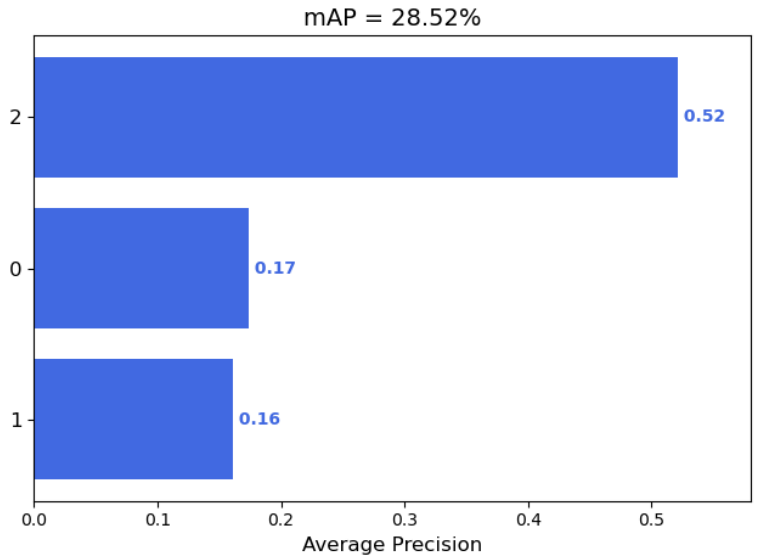
YOLOv4 all 33%의 mAP



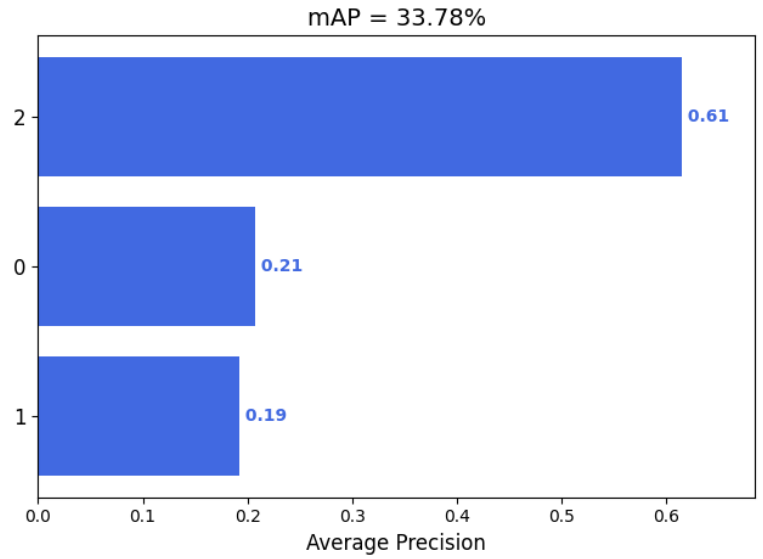
YOLOv4 all의 mAP

Detection

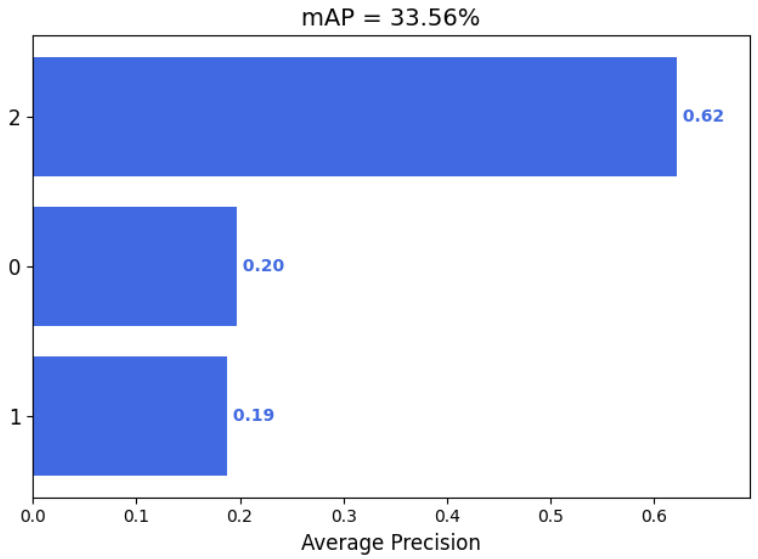
- YOLOv3 33%, YOLOv3 all 33%, YOLOv3 all
  - ➢YOLOv3는 YOLOv4에 비해 모든 기법을 다 적용했을 때 데이터의 수에 관련하여 성능차이가 크지 않음



YOLOv3 33%의 mAP



YOLOv3 all 33%의 mAP



YOLOv3 all의 mAP

Tracking

- MOTA
  - ➢모델별-비디오별 mean MOTA

	yolov3	yolov3_33	yolov4	yolov4_33	yolov4_anchor150	yolo4_33_aug	yolov4_33_608	yolov4_33_ciou	yolov4_33_focalx	yolo4_all	yolo4_all2	yolov3_all	yolov3_all2
uav0000201_00000_v	0.19915	0.12911	0.04186	0.00000	0.16719	-0.38713	0.10691	0.11280	0.13712	0.11289	-0.03846	-0.13222	0.02550
uav0000297_00000_v	0.15344	0.01709	-0.03418	0.00000	-0.06079	-0.48383	-0.05866	-0.06283	0.03661	0.09954	-0.18879	-0.26066	0.04681
uav0000120_04775_v	0.08873	-0.02509	-0.16852	0.00000	-0.02703	-0.17979	0.02661	-0.00327	-0.05532	-0.06696	-0.00713	-0.35566	-0.01282
uav0000249_00001_v	0.30917	0.28739	0.05423	0.00000	0.31814	-0.86314	0.20135	0.27789	0.21811	0.20967	0.34824	0.29209	0.29721
uav0000073_00600_v	-0.03471	-0.00034	-0.74207	-0.67074	-0.26099	-0.55560	0.10013	-0.37810	-0.11018	0.01134	-0.15047	-0.15828	0.08675
uav0000355_00001_v	0.08786	0.11987	0.03715	-0.78226	0.17508	-0.43702	0.03837	0.07774	0.12527	0.14228	0.02756	0.15193	0.06555
uav0000088_00290_v	0.08768	0.01439	-0.11495	-0.42118	0.02566	-0.33926	0.08267	-0.21341	-0.05330	0.03825	0.15180	-0.09529	0.10339
uav0000161_00000_v	0.11479	0.04469	-0.02812	-0.79242	0.03717	-0.23244	0.13502	0.03734	0.07191	0.12025	0.03256	0.05748	0.15117
uav0000370_00001_v	0.13623	-0.01492	0.03845	-8.26185	-5.52602	-2.72887	0.16274	0.10109	-3.59496	0.02287	0.10805	-0.01558	-0.02618
uav0000297_02761_v	0.13720	-0.01706	0.04199	-0.89844	-0.11992	-0.26040	0.09789	-0.20293	-0.07069	0.03144	-0.20394	-0.35608	0.10860
uav0000009_03358_v	0.33579	0.32692	-0.02025	0.07873	0.31405	-0.59027	0.32339	0.18320	0.13760	0.14003	0.24521	0.27912	0.22135
uav0000188_00000_v	0.05549	-0.07582	-0.15925	-0.33501	0.04026	-0.38794	-0.05297	0.03422	0.00048	-0.02370	0.00171	0.03370	-0.03436
uav0000119_02301_v	0.14345	0.08005	-0.04495	-0.73044	0.04745	-0.83588	0.12472	0.10461	0.10252	-0.02026	-0.07339	-0.02927	0.06632
uav0000073_04464_v	0.01618	-0.04886	-0.18934	-0.24478	-0.04609	-0.23760	0.07289	-0.18077	-0.05691	-0.10702	-0.23085	-0.29534	0.02339
uav0000306_00230_v	0.06735	-0.05580	-0.02055	-0.91210	-0.01421	-0.51192	-0.15816	0.00999	0.12471	-0.13706	-0.22713	-0.18690	-0.06282
uav0000077_00720_v	0.28736	0.28201	0.07285	0.03107	0.29235	-0.42605	0.17465	0.15620	0.17143	0.20011	0.18634	0.00135	0.16041
uav0000249_02688_v	0.20219	0.19037	0.00610	-0.65485	0.23454	0.10091	0.10091	0.17021	0.12778	0.12915	0.08921	0.14632	0.12268

Tracking

- MOTA
  - ➢Augmentation은 어두운 비디오를 더 잘 확인하는가?

	yolov3	yolov3_33	yolov4	yolov4_33	yolov4_anchor150	yolo4_33_aug	yolov4_33_608	yolov4_33_ciou	yolov4_33_focalx	yolo4_all	yolo4_all2	yolov3_all	yolov3_all2
uav0000201_00000_v	0.19915	0.12911	0.04186	0.00000	0.16719	-0.38713	0.10691	0.11280	0.13712	0.11289	-0.03846	-0.13222	0.02550
uav0000297_00000_v	0.15344	0.01709	-0.03418	0.00000	-0.06079	-0.48383	-0.05866	-0.06283	0.03661	0.09954	-0.18879	-0.26066	0.04681
uav0000120_04775_v	0.08873	-0.02509	-0.16852	0.00000	-0.02703	-0.17979	0.02661	-0.00327	-0.05532	-0.06696	-0.00713	-0.35566	-0.01282
uav0000249_00001_v	0.30917	0.28739	0.05423	0.00000	0.31814	-0.86314	0.20135	0.27789	0.21811	0.20967	0.34824	0.29209	0.29721
uav0000073_00600_v	-0.03471	-0.00034	-0.74207	-0.67074	-0.26099	-0.55560	0.10013	-0.37810	-0.11018	0.01134	-0.15047	-0.15828	0.08675
uav0000355_00001_v	0.08786	0.11987	0.03715	-0.78226	0.17508	-0.43702	0.03837	0.07774	0.12527	0.14228	0.02756	0.15193	0.06555
uav0000088_00290_v	0.08768	0.01439	-0.11495	-0.42118	0.02566	-0.33926	0.08267	-0.21341	-0.05330	0.03825	0.15180	-0.09529	0.10339
uav0000161_00000_v	0.11479	0.04469	-0.02812	-0.79242	0.03717	-0.23244	0.13502	0.03734	0.07191	0.12025	0.03256	0.05748	0.15117
uav0000370_00001_v	0.13623	-0.01492	0.03845	-8.26185	-5.52602	-2.72887	0.16274	0.10109	-3.59496	0.02287	0.10805	-0.01558	-0.02618
uav0000297_02761_v	0.13720	-0.01706	0.04199	-0.89844	-0.11992	-0.26040	0.09789	-0.20293	-0.07069	0.03144	-0.20394	-0.35608	0.10860
uav0000009_03358_v	0.33579	0.32692	-0.02025	0.07873	0.31405	-0.59027	0.32339	0.18320	0.13760	0.14003	0.24521	0.27912	0.22135
uav0000188_00000_v	0.05549	-0.07582	-0.15925	-0.33501	0.04026	-0.38794	-0.05297	0.03422	0.00048	-0.02370	0.00171	0.03370	-0.03436
uav0000119_02301_v	0.14345	0.08005	-0.04495	-0.73044	0.04745	-0.83588	0.12472	0.10461	0.10252	-0.02026	-0.07339	-0.02927	0.06632
uav0000073_04464_v	0.01618	-0.04886	-0.18934	-0.24478	-0.04609	-0.23760	0.07289	-0.18077	-0.05691	-0.10702	-0.23085	-0.29534	0.02339
uav0000306_00230_v	0.06735	-0.05580	-0.02055	-0.91210	-0.01421	-0.51192	-0.15816	0.00999	0.12471	-0.13706	-0.22713	-0.18690	-0.06282
uav0000077_00720_v	0.28736	0.28201	0.07285	0.03107	0.29235	-0.42605	0.17465	0.15620	0.17143	0.20011	0.18634	0.00135	0.16041
uav0000249_02688_v	0.20219	0.19037	0.00610	-0.65485	0.23454	0.10091	0.10091	0.17021	0.12778	0.12915	0.08921	0.14632	0.12268

Tracking

- 1DF1
  - ➢Augmentation은 어두운 비디오를 더 잘 확인하는가?

	yolov3	yolov3_33	yolov4	yolov4_33	yolov4_anchor150	yolo4_33_aug	yolov4_33_608	yolov4_33_ciou	yolov4_33_focalx	yolov3_all	yolov3_all2	yolo4_all	yolo4_all2
uav0000201_00000_v	0.5004	0.4640	0.3688	0.0000	0.4500	0.0012	0.4828	0.4326	0.3724	0.4088	0.4718	0.4426	0.4595
uav0000297_00000_v	0.4903	0.4447	0.3994	0.0000	0.4175	0.0010	0.4580	0.4367	0.4132	0.4093	0.4825	0.4696	0.4280
uav0000120_04775_v	0.4389	0.4085	0.3254	0.0000	0.4222	0.0000	0.4518	0.4214	0.3283	0.4196	0.4690	0.4434	0.4907
uav0000249_00001_v	0.6202	0.6005	0.5111	0.0000	0.5869	0.0093	0.5909	0.5899	0.5026	0.6244	0.6259	0.5804	0.6539
uav0000073_00600_v	0.1693	0.3437	0.2154	0.1675	0.3220	0.0030	0.3489	0.2567	0.2732	0.4043	0.4025	0.3755	0.3609
uav0000355_00001_v	0.4010	0.3894	0.3774	0.0018	0.4055	0.0001	0.4469	0.3927	0.3179	0.4393	0.4464	0.4075	0.4068
uav0000088_00290_v	0.3185	0.4383	0.3573	0.3464	0.4600	0.0000	0.3983	0.3329	0.3204	0.4589	0.4476	0.4756	0.5208
uav0000161_00000_v	0.3852	0.4017	0.3455	0.0085	0.3732	0.0007	0.4623	0.3736	0.3131	0.4610	0.4648	0.4092	0.4587
uav0000370_00001_v	0.3456	0.3452	0.3695	0.0000	0.0000	0.0014	0.3806	0.3711	0.0009	0.3408	0.3313	0.3428	0.3798
uav0000297_02761_v	0.3670	0.3223	0.3487	0.0020	0.3214	0.0002	0.4456	0.3609	0.3023	0.3616	0.4602	0.4062	0.4084
uav0000009_03358_v	0.6040	0.5819	0.3164	0.4076	0.5668	0.0015	0.6114	0.4813	0.4004	0.5962	0.5389	0.4727	0.5727
uav0000188_00000_v	0.3419	0.3131	0.2270	0.0011	0.2860	0.0033	0.3370	0.3182	0.1811	0.3955	0.3706	0.3508	0.4164
uav0000119_02301_v	0.2752	0.2831	0.2036	0.0108	0.1962	0.0005	0.3551	0.2230	0.2951	0.4679	0.4736	0.4370	0.4708
uav0000073_04464_v	0.2150	0.2606	0.1500	0.1262	0.2511	0.0003	0.2943	0.2620	0.1802	0.2916	0.3265	0.2776	0.2855
uav0000306_00230_v	0.5457	0.5190	0.4987	0.0007	0.5106	0.0001	0.4802	0.5214	0.5040	0.4922	0.5035	0.4821	0.4799
uav0000077_00720_v	0.5454	0.5589	0.4811	0.4267	0.5549	0.0018	0.5340	0.5000	0.4575	0.5233	0.5618	0.5841	0.5719
uav0000249_02688_v	0.5628	0.5554	0.4573	0.0042	0.5598	0.5563	0.5563	0.5340	0.4632	0.5754	0.5766	0.5535	0.5710

Tracking

- 1DF1
  - ➢Anchor 150, input 608, Ciou 방법과 Crowded와 의 연관성은?

	yolov3	yolov3_33	yolov4	yolov4_33	yolov4_anchor150	yolo4_33_aug	yolov4_33_608	yolov4_33_ciou	yolov4_33_focalx	yolov3_all	yolov3_all2	yolo4_all	yolo4_all2
uav0000201_00000_v	0.5004	0.4640	0.3688	0.0000	0.4500	0.0012	0.4828	0.4326	0.3724	0.4088	0.4718	0.4426	0.4595
uav0000297_00000_v	0.4903	0.4447	0.3994	0.0000	0.4175	0.0010	0.4580	0.4367	0.4132	0.4093	0.4825	0.4696	0.4280
uav0000120_04775_v	0.4389	0.4085	0.3254	0.0000	0.4222	0.0000	0.4518	0.4214	0.3283	0.4196	0.4690	0.4434	0.4907
uav0000249_00001_v	0.6202	0.6005	0.5111	0.0000	0.5869	0.0093	0.5909	0.5899	0.5026	0.6244	0.6259	0.5804	0.6539
uav0000073_00600_v	0.1693	0.3437	0.2154	0.1675	0.3220	0.0030	0.3489	0.2567	0.2732	0.4043	0.4025	0.3755	0.3609
uav0000355_00001_v	0.4010	0.3894	0.3774	0.0018	0.4055	0.0001	0.4469	0.3927	0.3179	0.4393	0.4464	0.4075	0.4068
uav0000088_00290_v	0.3185	0.4383	0.3573	0.3464	0.4600	0.0000	0.3983	0.3329	0.3204	0.4589	0.4476	0.4756	0.5208
uav0000161_00000_v	0.3852	0.4017	0.3455	0.0085	0.3732	0.0007	0.4623	0.3736	0.3131	0.4610	0.4648	0.4092	0.4587
uav0000370_00001_v	0.3456	0.3452	0.3695	0.0000	0.0000	0.0014	0.3806	0.3711	0.0009	0.3408	0.3313	0.3428	0.3798
uav0000297_02761_v	0.3670	0.3223	0.3487	0.0020	0.3214	0.0002	0.4456	0.3609	0.3023	0.3616	0.4602	0.4062	0.4084
uav0000009_03358_v	0.6040	0.5819	0.3164	0.4076	0.5668	0.0015	0.6114	0.4813	0.4004	0.5962	0.5389	0.4727	0.5727
uav0000188_00000_v	0.3419	0.3131	0.2270	0.0011	0.2860	0.0033	0.3370	0.3182	0.1811	0.3955	0.3706	0.3508	0.4164
uav0000119_02301_v	0.2752	0.2831	0.2036	0.0108	0.1962	0.0005	0.3551	0.2230	0.2951	0.4679	0.4736	0.4370	0.4708
uav0000073_04464_v	0.2150	0.2606	0.1500	0.1262	0.2511	0.0003	0.2943	0.2620	0.1802	0.2916	0.3265	0.2776	0.2855
uav0000306_00230_v	0.5457	0.5190	0.4987	0.0007	0.5106	0.0001	0.4802	0.5214	0.5040	0.4922	0.5035	0.4821	0.4799
uav0000077_00720_v	0.5454	0.5589	0.4811	0.4267	0.5549	0.0018	0.5340	0.5000	0.4575	0.5233	0.5618	0.5841	0.5719
uav0000249_02688_v	0.5628	0.5554	0.4573	0.0042	0.5598	0.5563	0.5563	0.5340	0.4632	0.5754	0.5766	0.5535	0.5710

느낀점

느낀점

- 프로젝트 수행 중 어려움 극복 사례
  - 고민수:
    - 데이터 제너레이터를 만드는 과정에서 증강을 사용하기 위해 Albumentation 라이브 러리를 사용하였는데 bounding box가 이미지 크기보다 벗어나는 문제를 알아차리고 고치는데 일주일이 걸렸다. albumentation/core/bbox_util.py를 수정하면 된다ㅎ
  - 손승우:
    - 실험을 돌려볼 때 시간적으로 여유가 없어서 새벽에도 일어나서 확인해 보고 다시 돌리고 잠이 너무 부족해지는 문제가 있었음. 코드를 좀 더 자율적으로 만들어보려고 수정해서 프로젝트 발표 전전날엔 확인만 종종 하면 될 수 있게 만들어서 해결했습니다. 전 전날이긴 하지만
  - 정민교:
    - 평가지표를 뽑기 위해 매트릭의 파이썬 코드를 한줄한줄 분석해 의미를 평가지표의 의미를 파악

느낀점

- 프로젝트에서 잘한 부분
  - 고민수:
    - 팀 프로젝트를 진행하면서 팀으로서 움직이거나 많은 시간을 프로젝트에 쏟지 못하는 상황에서 승우 님과 민교 님께 미안한 부분이 너무 많았다. 하지만 프로젝트를 그만두는 것은 더 안 좋은 선택일 것이라 생각하고 끝까지 진행하였는데 잘한 것 같다.
  - 손승우:
    - 맡은 바 임무를 다한 것. 어떠한 함수에 들어가는 input값이 뭔지, output값이 어떤 건지 파악하고 detector 모델로 bytetrack과 deepsort를 붙여볼 수 있었습니다.
  - 정민교:
    - 평가지표에 대한 논문분석 및 코드 분석

느낀점

- 프로젝트에서 아쉬운 부분
  - 고민수:
    - Detection 모델 구현에 너~무 오랜 시간이 걸린 것.. 이미 tracker는 다 되었는데 detection이 안돼서 기다리는 팀원들을 보고 있는 것이 미안하고, 또 더 다양한 실험과 그 결과를 분석할 충분한 시간이 없었다는 점이 아쉽다.
  - 손승우:
    - 평가 지표 점수가 좀 많이 낮은데 어떠한 이유 때문인지 잘 모르겠다는 거. 지금 프로젝트 마지막 날이 끝이 아니라 다음에 더 분석하고 연구해 봐야겠다 생각했습니다.
  - 정민교:
    - 초반의 의욕에 비해 후반에 이루어낸 것이 없고 건강상의 이슈로 남들만큼 열정적으로 참여 못한 점

SIAmese

Multi Object Tracking on “Visdrone” dataset

Ablation study on YOLO performance in multi object tracking

감사합니다.

Tracking

- 1DF1
  - ➢Anchor 150, input 608, Ciou 방법과 Crowded와 의 연관성은?